KTH ROYAL INSTITUTE
OF TECHNOLOGY

Degree Project in ICT Innovation

Second cycle, 30 credits

JVM Tuning with Machine Learning on Garbage Collection Logs

YAGMUR EREN

Stockholm, Sweden, 2024

JVM Tuning with Machine Learning on Garbage Collection Logs

YAGMUR EREN

Date: July 4, 2024

Supervisors: Poonam Parhar, Marina Shimchenko, Gizem Caylak
Examiner: David Broman
School of Electrical Engineering and Computer Science
Host company: Oracle
Swedish title: JVM-tuning med maskininlärning på skräpsamling-loggar

© 2024 Yagmur Eren

Abstract

The performance of Java applications is crucial for developers and system administrators. Performance is primarily assessed through throughput (work done in a unit of time) and latency (responsiveness) of the application. These depend not only on the efficiency of the written code but also on the performance of the language runtime, the Java Virtual Machine (JVM). Optimizing the performance of the JVM or adjusting the JVM flags for each application can be a tedious task because it requires significant expertise and effort. This project proposes a tuning strategy for specific memory-related JVM flags based on Machine Learning (ML) techniques, using information collected from the Garbage Collection (GC) logs where GC is a process of reclaiming unused memory during runtime in JVM. We explore the possibility of improving throughput while maintaining an acceptable application latency by using the JVM flag values suggested by the ML models for one Java program at a time. DaCapo and SPECjbb2015 benchmarks are the programs that we use to measure performance. We analyze the impact of different ML techniques on performance improvements. According to our findings, tuning the selected JVM flags using suggestions from the models shows considerable potential to enhance throughput by up to 20% compared to default flag settings while maintaining latency within acceptable limits. This project aims to help developers and system administrators automatically determine optimum values for memory-related JVM flags, ultimately saving time and effort to achieve better throughput without compromising latency.

Keywords

Java Virtual Machine, Machine Learning, Auto-tuning, JVM flags, Garbage Collection logs, Garbage First, Performance Optimization

Sammanfattning

Prestandan hos Java-applikationer är avgörande för utvecklare och systemadministratörer. Prestanda bedöms främst genom genomströmning (arbete utfört under en tidsenhet) och latens (responsivitet) hos applikationen. Dessa beror inte bara på effektiviteten hos den skrivna koden utan också på prestandan hos exekveringsmiljön, den s.k. JVM. Att optimera prestandan hos JVM eller justera JVM-flaggor för varje applikation kan vara en tidskrävande uppgift eftersom det kräver betydande expertis och ansträngning. Detta projekt föreslår en strategi för justering av specifika minnesrelaterade JVM-flaggor baserat på maskininlärnings-tekniker, med hjälp av information som samlats in från GC-loggar där GC är en process för att återta oanvänt minne under körning i JVM. Vi utforskar möjligheten att förbättra genomströmningen samtidigt som vi bibehåller en acceptabel applikationslatens genom att använda de flaggvärden som föreslås av maskininlärning-modellerna för ett Java-program åt gången. DaCapo och SPECjbb2015 benchmarkprogrammen är de program som vi använder för att mäta prestanda. Vi analyserar inverkan av olika maskininlärnings-tekniker på prestandaförbättringar. Enligt våra resultat visar justering av de valda JVM-flaggorna med hjälp av förslag från modellerna betydande potential att förbättra genomströmningen med upp till 20% jämfört med standardflagginställningar samtidigt som latensen hålls inom acceptabla gränser. Detta projekt syftar till att hjälpa utvecklare och systemadministratörer att automatiskt bestämma optimala värden för minnesrelaterade JVM-flaggor, vilket slutligen sparar tid och ansträngning för att uppnå bättre genomströmning utan att kompromissa med latensen.

Nyckelord

Java Virtual Machibe, Maskininlärning, Automatisk justering, JVM-flaggor, Garbage Collection-loggar, Garbage First, Prestandaoptimering

Acknowledgments

First, I am profoundly grateful to my supervisors, Poonam Parhar and Marina Shimchenko, for their mentorship, insightful advice, and continuous encouragement, for patiently answering all my questions, and for this amazing opportunity. My sincere thanks to my academic supervisor, Gizem Caylak, for all her support and valuable suggestions during this process. I am also thankful to my examiner, David Broman, for his thorough evaluation and valuable comments. I also would like to thank all my colleagues at Oracle for their friendship and assistance. Finally, my deepest thanks to my boyfriend and my family for their love, emotional support, and encouragement during this journey.

Stockholm, July 2024
Yagmur Eren

Contents

1 Introduction **1**
1.1 Preliminary Background 1
1.2 Problem and Motivation 2
1.2.1 Research Question 2
1.2.2 Research Process 3
1.2.3 Scientific and engineering issues 3
1.3 Purpose 3
1.4 Contributions 4
1.5 Delimitations 4
1.6 Structure of the thesis 5

2 Background **7**
2.1 JVM and GC 7
2.1.1 G1 Garbage Collector 8
2.1.2 GC logs of G1 8
2.1.3 Harnessing GC logs: Extracting Meaningful Information 8
2.2 Choice of JVM flags to be tuned 10
2.3 ML models 11
2.4 Related work 13
2.4.1 Auto-tuning the Java Virtual machine 13
2.4.2 OneStopTuner: An End to End Architecture for JVM Tuning of Spark Applications 13
2.4.3 Predicting JVM Parameters for Performance Tuning Using Different Regression Algorithms 14
2.4.4 Automated Tuning of the JVM with Bayesian Optimization 14

2.4.5 The Taming of the Shrew: Increasing Performance by Automatic Parameter Tuning for Java Garbage Collectors . . . 14
2.4.6 Garbage Collection Auto-tuning for Java MapReduce on Multi-cores . . . 15
2.4.7 JVM Configuration Management and Its Performance Impact for Big Data Applications . . . 16
2.4.8 CGPTuner: a Contextual Gaussian Process Bandit Approach for the Automatic Tuning of IT Configurations Under Varying Workload Conditions . . . 16
2.4.9 Optimization of Java Virtual Machine Flags using Feature Model and Genetic Algorithm . . . 17
2.4.10 Summary . . . 18

3 Design and Implementation 19
3.1 Project Workflow . . . 19
3.2 Pipeline Design . . . 21
3.2.1 Feature Engineering . . . 21
3.2.2 ML models . . . 26
3.2.3 Inference Data for Predictions . . . 27
3.2.4 Model Predictions and Mapping to Flag Values . . . 28

4 Evaluation and Results 30
4.1 Experimental Configuration . . . 30
4.1.1 Hardware/Software Setup . . . 31
4.1.2 Reliability and Validity of the Results . . . 31
4.2 Evaluation Method . . . 31
4.3 Results . . . 33
4.3.1 Comparison of the Default and Tuned Runs of the Benchmarks . . . 33
4.3.2 Comparison of the Overall Performance of ML Models 42
4.4 Discussion . . . 43

5 Conclusion 46
5.1 Future Work . . . 46
5.2 Conclusion . . . 47

References 49

A Supporting materials **55**
A.1 Example GC Log Analysis 55
A.1.1 Example GC Log 55
A.1.2 Example Regular Expressions Used to Extract Features From GC Logs 58
A.1.3 Extracted Features From GC Log in Tabular Format . 58
A.1.4 Data in Tabular Format After Processing 60
A.2 Young Generation Size Graphs 62
A.3 Suggested Flag Values 64
A.4 Benchmark Execution Commands 64
A.5 The Five Largest Pause Times of Tuned SPECjbb2015 Executions 65

List of Figures

1.1 Research process 3

2.1 Example GC log 9
2.2 The choice of JVM flags to be auto-tuned 11
2.3 The effects of different flag values on the execution time of an application 12

3.1 Project workflow 19
3.2 Representation of the time window after sliding once 25
3.3 Representation of the dataset after processing with time window approach 26

4.1 Results of SPECjbb2015 benchmark 34
4.2 The graphs of maximum GC pause times in local time windows for SPECjbb2015 36
4.3 Results of DaCapo/cassandra benchmark 38
4.4 The graphs of maximum GC pause times in local time windows for DaCapo/cassandra 39
4.5 Results of DaCapo/h2 benchmark 41

A.1 Graph of SPECjbb2015 young generation size over time 62
A.2 Graph of DaCapo/cassandra young generation size over time . 63
A.3 Graph of DaCapo/h2 young generation size over time 63

List of Tables

3.1 The correlation values between the features and the label . . . 27

4.1 Java benchmarks 32
4.2 The five highest maximum GC pause times 37
4.3 Improvements for SPECjbb2015 43
4.4 Improvements for DaCapo/cassandra 44
4.5 Improvements for DaCapo/h2 44

A.1 An example portion of extracted features from GC log 59
A.2 An example portion of processed dataset 61
A.3 Suggested flag values by ML models for each benchmark . . . 64
A.4 The five highest maximum GC pause times 65

Listings

A.1 Command to run SPECjbb2015 benchmark 64
A.2 Command to run DaCapo/cassandra benchmark 64
A.3 Command to run DaCapo/h2 benchmark 65

List of acronyms and abbreviations

CMS	Concurrent Mark-Sweep
ET	Extra Trees
G1	Garbage First
GB	Gradient Boosting
GC	Garbage Collection
GPR	Gaussian Process Regressor
JDK	Java Development Kit
jOPS	Java Operations Per Second
JRE	Java Runtime Environment
JVM	Java Virtual Machine
ML	Machine Learning
OPS	Operations Per Second
OS	Operating System
RFR	Random Forest Regression
SDG	Sustainable Development Goal
SVR	Support Vector Regression

Chapter 1

Introduction

As Oracle states [1], there are more than 60 billion installations of Java Virtual Machine (JVM) globally. It has various run-time flags [2] that the developers and system administrators can set to enhance program performance, namely the throughput and the latency. In scenarios such as batch processing, stream processing, and big data applications, a high number of processed work over a unit of time, i.e., high throughput, is required. When a quick response time is necessary for the user experience or system functionality such as online gaming, high-frequency trading systems, and web applications, i.e., low latency is desirable. The developers and system administrators can achieve better throughput and latency by carefully adjusting the JVM flags. However, this laborious task requires a deep understanding of parameters to identify the optimal values, which consumes significant time and effort. JVM tuning with Machine Learning (ML) on Garbage Collection (GC) logs project focuses on learning and understanding the behavior of a Java application from its GC logs and building an ML model capable of suggesting optimal JVM flag values to improve runtime performance. This chapter provides a brief background to understand this project and describes the problem, purpose, and research outcomes. Finally, the delimitations are explained and we outline the structure of this thesis document.

1.1 Preliminary Background

JVM is a crucial component of the Java platform which provides the hardware and operating system independence for the Java program [3]. There are several concepts to be comprehended to carry out this project. This section provides a concise overview of the relevant concepts, while a more comprehensive

explanation can be found in Chapter 2.

GC involves reclaiming memory occupied by the unused objects (garbage) that are unreachable from the root objects [4]. This process, also known as automatic memory management done by garbage collectors, where garbage collectors automatically search memory for unused objects and free up allocated memory [4]. In this project, we employ the Garbage First (G1) [5] garbage collector, the default option in the Java Development Kit (JDK) [6]. Each garbage collector algorithm operates uniquely, resulting in different formats for the generated GC logs. Understanding the chosen garbage collector's operation is crucial for interpreting these logs accurately. Furthermore, the JVM tuning process varies accordingly. The GC logs can be collected from the Java applications by configuring the necessary settings.

The runtime performance of JVM depends heavily on GC activities and we can obtain log files where the GC events-related information is collected. In this project, we focus on this information from GC logs to understand the program's runtime behavior. We train ML models with this information to configure JVM options to improve the Java application's runtime performance. ML models can capture the relationship between the extracted information from GC logs and the program's performance. In this research, we focus on auto-tuning the JVM by finding the optimal usage of the young generation region, which is where the Java objects are initially stored within the heap region. We give more details on these concepts in Chapter 2.

1.2 Problem and Motivation

The main research problem of this thesis is to investigate whether utilizing the suggested JVM flag values by a ML model, trained and validated with data extracted from the GC log of a specific Java application, can lead to runtime performance improvements of the application.

1.2.1 Research Question

The following question is considered to evaluate the findings from our research project:

> To what extent is it possible to improve the performance of a Java application by using the suggested JVM flag values by an ML model that has been trained and validated with the information obtained from a GC log of that particular application?

1.2.2 Research Process

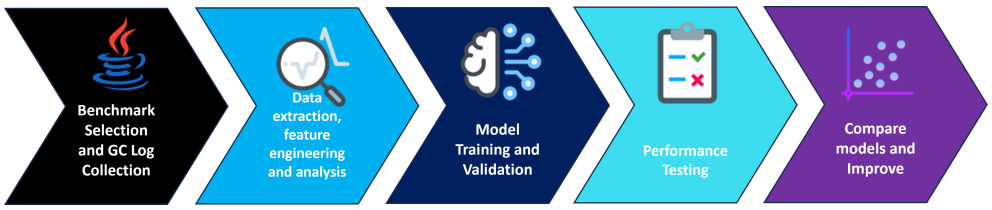


Figure 1.1: Research process: The main steps taken during the project

In this project, we follow a series of steps outlined in Figure 1.1. We begin with selecting Java benchmarks and collecting GC logs from them as raw data for our analysis. Next, we engineer and analyze the features that we extracted from the raw data. Once we have pre-processed the data, we train and validate the selected ML models. Using the recommendations of the resultant models, we map them to relevant JVM flags and re-run the selected benchmark to observe the performance changes in the application. Finally, we compare different models based on the changes observed between the original run and the tuned runs with suggested configurations.

1.2.3 Scientific and engineering issues

Scientific issues

- Acquiring sufficient data points from the GC log of a single program execution.
- Choosing suitable ML models capable of delivering precise recommendations using the data gathered from GC logs

Engineering issues

- Design and implementation of the ML pipeline
- Mapping the extracted features from GC logs and the JVM flags to be able to provide a flag value suggestion

1.3 Purpose

The primary objective of this work is to be able to auto-tune JVM based on the information we can get from the GC log of a single execution of

Java applications so that we can contribute to the applications' performance improvement process conducted by the developers and system administrators. This project does not raise any ethical concerns since it primarily focuses on only the technical aspects of performance optimization of Java applications. This project aims to contribute to having efficient and well-optimized software applications that may help reduce resource consumption, lower energy usage, and improve overall system performance. That way, it will contribute to one of the goals of United Nations Sustainable Development Goal (SDG) 9: Industry, Innovation, and Infrastructure [7].

1.4 Contributions

This research makes the following contributions to the field:

- A novel method for auto-tuning the JVM based on training an ML model by leveraging the information from the GC log of an application. This approach stands out from existing studies as it provides tuning suggestions based on a single application run instead of multiple runs.
- GC logs have valuable information about the nature of the application. Uniquely, we process this data by segmenting it into small time windows and evaluating the performance of each window. This strategy enables the models to capture the relationship between features and application performance.

1.5 Delimitations

The scope of this study is delimited by the following key factors that define its boundaries and limitations that can be addressed in future work:

> Since this project is only a 20-week work as a master thesis, including both report writing, the resultant ML pipeline of this project targets to work for one Java program at a time. We use a GC log of that particular program to train the ML model that provides the flag value suggestions. A study that can provide suggested flag values for various Java programs would be more comprehensive where only one pre-trained ML model with a larger dataset (at least half a million data points) of GC logs collected from various applications.

The JVM comes with a large number of flags. We chose to focus on two flags (see Section 2.2) that are tuned for this project. The GC logs provide beneficial insights about the young generation usage of the JVM during runtime, which leads us to consider that these particular young generation size related flags can be tuned by our pipeline based on those insights. Section 2.2 explains the details of the JVM flag selection. It is worth noting that even auto-tuning these two flags is a significant task, as finding their optimal values for best performance is non-trivial. These flags can take integer values from 0 to 100, representing percentage values, and determining the ideal setting requires careful consideration and testing.

There are several garbage collector algorithms in the JVM – G1, ZGC, Concurrent Mark-Sweep (CMS), ParallelGC and SerialGC. The desired garbage collector can be enabled with flags such as `-XX:+UseG1GC`, `XX:+UseZGC`, `-XX:+UseSerialGC` [8]. Each garbage collector type provides different formatted log files that require different parsing. Considering this, we only stick to the default garbage collector in the JVM version we use, that is, G1. It is explained in Section 2.1.1.

We test the pipeline and collect the results based on SPECjbb2015 and DaCapo (h2 and cassandra) benchmarks.

For statistical significance, the results should be collected based on multiple runs of the benchmarks. However, considering time and resource restrictions, we keep runs within a reasonable number. DaCapo benchmarks typically require less time to execute compared to SPECjbb2015. Therefore, we run the DaCapo benchmarks 20 times, while the SPECjbb2015 benchmarks run 5 times.

1.6 Structure of the thesis

The thesis report is outlined as the following:

- Chapter 2 presents relevant background knowledge required for the context of this project and previous related works in the research field of auto-tuning the JVM.
- Chapter 3 describes the project workflow and our design.

- Chapter 4 contains the evaluation and results of this project's findings and discusses the experimental results.
- Chapter 5 summarizes the conclusions from this thesis and outlines future work.

Chapter 2

Background

This chapter provides background information about the JVM, GC events, and GC logs, focusing on G1 garbage collector. We also explain techniques to analyze the data, ML models that we use, and the state of the art in this field.

2.1 JVM and GC

JVM acts as the engine that converts Java programming instructions into bytecodes, which are then executed by the local Operating System (OS) [2]. Within JVM, the heap area is critical as it is where objects are stored and managed by several garbage collector algorithms such as G1, ZGC, CMS, ParallelGC and SerialGC. The responsibility of GC is to automatically free up memory by removing objects no longer in use. The generational garbage collection approach divides the Java heap into regions called young and old generations where the objects are stored depending on how long they have been alive [9]. The young generation consists of two regions: Eden and two Survivor regions (see Appendix A.2 to see how the young generation looks like in different programs before and after tuning). Eden region is the part of the Java heap where the objects are stored when they are created for the first time and if they manage to survive some cycles of GC, they are moved to survivor regions and then, to the old generation region. There are mainly three GC events: minor GC, major GC and full GC. The minor collection is the GC collection within the young generation and is triggered when the Eden region is full. The major collections collect garbage in the old generation and finally, both young and old generation memories are reclaimed when the heap is full, which is a full collection [9].

2.1.1 G1 Garbage Collector

The JDK version we use (see 4.1.1) has the G1 as the default garbage collector which has also been identified as the most efficient choice in the study by Kumar *et al.* [10]. These encouraged us to use G1 garbage collector in our project. The G1 garbage collector partitions memory into fixed-sized heap regions, including the young generation (Eden and survivor regions), the old generation, and the permanent generation. Unlike other types of garbage collectors, G1 prioritizes the regions with the most reclaimable space first, followed by those with the least live data, aiming to achieve a balance between throughput and latency [2].

In the context of G1, various types of GC pauses are observed, including normal, mixed, prepare mixed, and concurrent start pauses. According to Oracle's G1 tuning article [11], the GC cycle starts with a young (normal) pause that collects only the young generation and promotes the surviving objects to the old generation. A concurrent start pause occurs when the occupancy of the old generation reaches a certain threshold, during which concurrent marking takes place, followed by remark and cleanup phases. Subsequently, a space reclamation phase, also known as a mixed collection or pause, is initiated. The normal GC, remark and mixed phases are *stop-the-world* events [12]. In this project, we only consider the impact of *stop-the-world* events on program performance, not the concurrent pauses.

2.1.2 GC logs of G1

GC logs can provide valuable information about the nature of the application. JVM produces these logs as detailed records of the garbage collection process. By parsing these text files, we can extract important data such as the duration of GC events, usage of different memory areas before and after GC, the frequency of GC, and more. An example snippet from a GC log is shown in Figure 2.1, with different information highlighted. A larger example can be found in Appendix A.1.1. The file format may vary depending on the garbage collector type, these can be ParallelGC, SerialGC, G1, and ZGC.

2.1.3 Harnessing GC logs: Extracting Meaningful Information

In this project, we use GC logs that consist of a stream of time-stamped events detailing the GC activities and memory usage over time. We can effectively extract pertinent information from a given GC log by utilizing

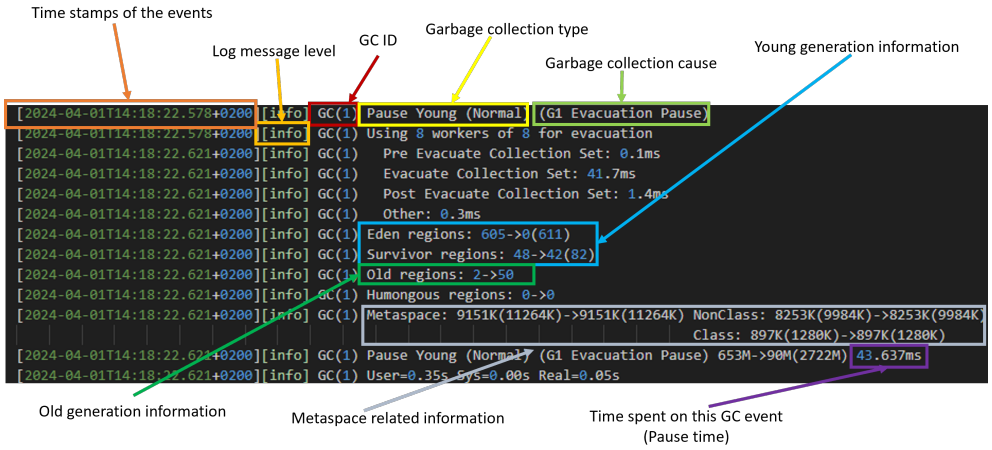


Figure 2.1: Example GC log: An example snippet from a G1 GC log

advanced regular expressions. An example of the extracted data from a GC log can be found in Appendix A.1.3. The relevant information, which we can extract through this process, includes the following:

- Time stamps: Time stamps are chronological indicators of when GC events or other memory management operations took place within a Java application. For GC logging with timestamps, We use the `-Xlog:gc*:file=<path_to_gc_log>:time` flag.
- GCID: ID of each GC event.
- Heap region: Heap is where the Java objects are created and stored. Heap size and usages before and after GC events are extracted.
- Young region: New objects are first stored in the young generational region, consisting of Eden and survivor regions. The sizes and their usages before and after GC events are extracted.
- Old region: If the objects can survive some time, they migrate to the old generational regions. The old generation size and usage before and after GC events are extracted.
- Metaspace: Metaspace is the native memory manager in JVM optimized for the low-overhead allocation of off-heap memory. However, it cannot easily delete arbitrary memory blocks [13]. Metaspace size and usage before and after GC are extracted.
- Pause time: The time spent for each GC event.

- Pause types: G1 has several GC pause types with different reasonings. These are concurrent start, mixed, normal, and prepare mixed.

2.2 Choice of JVM flags to be tuned

JVM comes with hundreds of flags. For this project, we chose to auto-tune two flags related to memory management because these parameters are particularly challenging to set and optimize manually. These flags are `-XX:G1MaxNewSizePercent` and `-XX:G1NewSizePercent`, which set the maximum and minimum size of the young generation in terms of percentages of heap size, respectively. Memory-related flags are notoriously difficult to configure due to the complexity of understanding an application's memory footprint, which is obscured by Java's automatic memory management from the developer. Even if one could find a suitable total memory allocation, determining the division between young and old generations relies on unpredictable runtime heuristics that are nearly impossible to figure out manually.

Figure 2.2 visualizes these flags as the upper and lower boundaries of young generation usages in the Java heap region. The shaded orange area under 60% represents that the maximum young generation size used during runtime will be up to that amount of the heap size and the shaded green area under 5% indicates that minimum of that amount of the heap will be used for the young generation. Upper and lower boundaries of the young generation size directly affect the minor GC events which are the collections in the young generation.

The two flags we want to auto-tune in this project can be given as such:

- `-XX:G1MaxNewSizePercent`: As given in the Java command options guide [8], the percentage of the Java heap size for the maximum size of the young generation usage is decided by this flag with the default value of 60%. Adjusting the maximum usage of the young generation helps us control the individual pause time for each minor GC event. This directly affects the responsiveness of the application. If we let JVM spend too much time during the minor GC events, the application will not be able to respond in a tolerable amount of time, resulting in poor latency.

- `-XX:G1NewSizePercent`: Unlike the previous flag, this flag is used to set the minimum percentage of the heap for young generation

usage and its default value is 5%. Increasing the value of minimum percentages of the young generation usage decreases the frequency of GC events that lead to a better number for Operations Per Second (OPS) metric, in other words, better throughput.

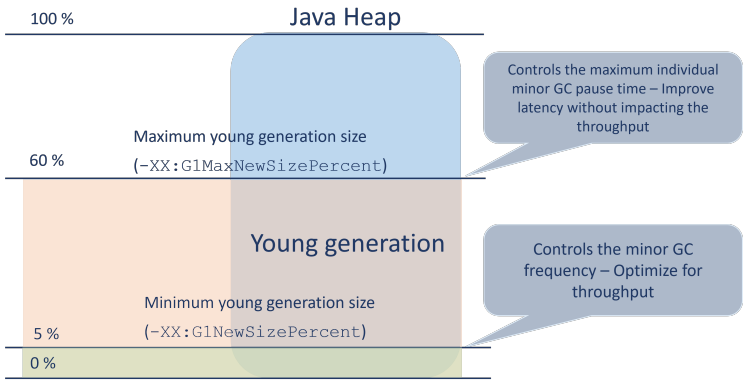


Figure 2.2: The choice of JVM flags to be auto-tuned: Representation of chosen flags to be tuned for this project and their effects on application performance. The blue area represents the Java heap. By default, up to 60% of the Java heap shown in the orange shaded area can be used for the young generation where this value can be changed by `-XX:G1MaxNewSizePercent` flag. The green shaded area shows the default 5% of the heap that is used for the young generation and can be changed by setting `-XX:G1NewSizePercent` flag.

By adjusting these flags, we can control the young generation size and manipulate the behavior of the GC events which directly impact the Java application performance. For instance, Figure 2.3 shows the variation in execution time of the DaCapo/h2 benchmark on `-XX:G1NewSizePercent` flag. The application execution time is a performance quality metric for this benchmark (see Section 4.2), indicating a better throughput.

In this research, we aim to find the optimal values of these flags for a given Java program, striking a balance between the upper and lower limits of the usage of the young generation within the heap region.

2.3 ML models

To explore the behavior of ML models on this project, we select a diverse range of methodologies. The models should be effective on small datasets

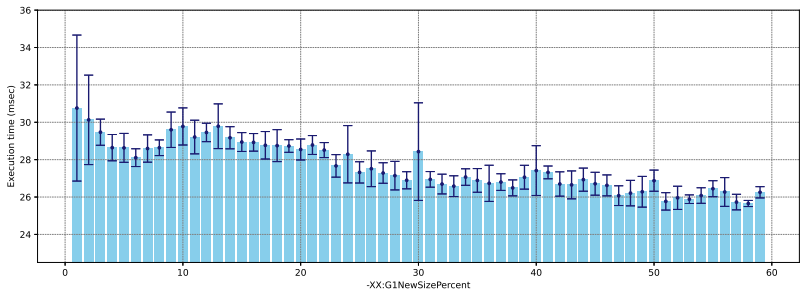


Figure 2.3: Example of the effects of different flag values on the execution time of an application: The change in total execution time of DaCapo/h2 benchmark based on various values of `-XX:G1NewSizePercent` flag.

(i.e., 100-2500 data samples) because we have one GC log from an application and they should handle potential non-linear relations and capture complex patterns such as the relation between the information we get from the GC log and the memory usage. The models we use in this project are Support Vector Regression (SVR) [14], Gradient Boosting (GB) [15], Random Forest Regression (RFR) [16], Extra Trees (ET) [17] and Gaussian Process Regressor (GPR) [18]. The performance of these models is compared and the results are presented in Chapter 4. The following list provides an explanation of the algorithms we chose and the reasons behind our selection:

- **SVR:** The idea behind SVR [14] is to capture the relationship between the variables by finding a hyperplane after converting the data into a higher dimensional space. It is a robust algorithm against outliers in the data and can handle non-linear relations effectively. Moreover, since it can maintain flexibility while capturing the complexities in the dataset, this model was a great candidate for our case.

- **Ensemble Methods:** For datasets with limited sample size, as in our case that comprises only one GC log produced by a Java application, it is recommended [19] to employ ensemble methods to enhance the predictive performance. Among ensemble methods, we pick GB [15], RFR [16] and ET [17]. They all aggregate multiple decision trees to construct a better predictor from weaker predictors. They are suitable algorithms for this project, considering their ability to capture complex patterns and well-suited for small datasets. They are also robust against overfitting.

- **GPR:** This model [18] is based on Gaussian processes [20], which

assume that any finite set of points in the input space has a joint Gaussian distribution. This allows the model to provide probabilistic predictions and quantify uncertainty.

These models come with a set of hyperparameters. For tuning these hyperparameters, we opt for Bayesian optimization, BayesSearchCV [21] function from scikit-learn. Unlike grid search, Bayes search does not try all the possible values. Instead, it evaluates the model with different parameter values and directs the search through the hyperparameter space, which may be more likely to have better performance by learning from these evaluations [22]. Once we have the best hyperparameters for each model, they are ready to be trained and tested.

2.4 Related work

2.4.1 Auto-tuning the Java Virtual machine

Jayasena *et al.* [23] explain their JVM performance optimization auto-tuning framework that can tune over 600 JVM flags by organizing them in a hierarchical structure which helps their tuner avoid generating invalid flag configurations. It organizes the flags hierarchically by categorizing them according to their compatibility together, e.g., the flags used for CMS garbage collector are ignored if ParallelGC is already set as the garbage collector algorithm. This novel structure helps to manage dependencies and reduce the flag configuration search space of the tuning. They obtain the optimal parameters based on many iterations of the applications with maximum duration of 200 minutes for tuning. The authors used SPECjvm2008 and DaCapo benchmarks and observed significant performance improvements in many benchmarks. They noted the improvements up to 42% with the mean of 26% for DaCapo benchmarks and mean of 19% for three SPECjvm2008 benchmarks.

2.4.2 OneStopTuner: An End to End Architecture for JVM Tuning of Spark Applications

Bindal *et al.* [24] introduce OneStopTuner, developed to optimize the performance of Spark applications by auto-tuning the JVM flags. It is a machine learning framework that utilizes active learning for data generation, picks the relevant JVM flag configurations with feature selection

algorithms, and applies Bayesian optimization for tuning. The authors claim that compared to traditional methods, OneStopTuner provides significant improvements in execution time (1.35 times speedup) and heap usage (50%).

2.4.3 Predicting JVM Parameters for Performance Tuning Using Different Regression Algorithms

Vijayakumar and Bharathi [25] represent a prediction of JVM parameters based on ML algorithms, specifically Huber, Tweedie, and SGD regressions. They train the models with approximately 400 GC logs generated from applications where parallel and serial garbage collectors are used. They collect multiple GC logs with a JVM flag configurations for heap size and GC type. By utilizing this GC log dataset, they train the regression models. Eventually, they report the accuracy of each model on predicting the necessary heap size, execution time, and the frequency of GC events. Their result shows that the Tweedie regression has the best accuracy among the others. This study does not address the problem of tuning the JVM. They are aiming to predict the future behavior of the JVM based on the necessary heap size for the application, its execution time and potential number of GC events during runtime.

2.4.4 Automated Tuning of the JVM with Bayesian Optimization

Twitter [26] discusses another optimization method of JVM performance based on Bayesian optimization. In their work, they mentioned that they replaced the manual job of performance engineers, that is, picking new parameters to test based on previous performance results obtained, which can be replaced with a tuning assistant to pick parameters in each iteration based on Bayesian optimization. They explain a case study demonstrating the efficacy of this approach in configuring JVM based micro-services, highlighting a significant decrease in manual tuning efforts.

2.4.5 The Taming of the Shrew: Increasing Performance by Automatic Parameter Tuning for Java Garbage Collectors

Lengauger and Mössenböck [27] propose automatic optimization of Java GC parameters to enhance application performance. They start by creating a

comprehensive parameter model that outlines all available GC parameters, their possible values, and constraints. An objective function is defined to measure the performance impact of any given set of parameters, focusing on minimizing the aggregated GC time. The optimization is carried out using an optimization algorithm named ParamILS, an iterated local search algorithm that iteratively refines parameter configurations by assessing improvements in the output of the objective function. This process is validated through experiments on DaCapo and SPECjbb2005 benchmarks and a real-world application, showing the effectiveness of the approach in significantly reducing GC times and, finally, improving overall application responsiveness and efficiency. This methodology offers a sophisticated practical solution to the complex and time-consuming task of manual GC tuning, by using algorithmic automation to navigate the vast parameter space efficiently.

2.4.6 Garbage Collection Auto-tuning for Java MapReduce on Multi-cores

Singer *et al.* [28] focus on optimizing GC for MapReduce Java (MRJ), a big data analysis model designed for multi-core architectures. The authors highlight the significant impact of GC on MRJ performance and scalability, particularly when deploying standard MapReduce benchmarks on a multi-core Intel Core i7 processor. They examine SerialGC, ParallelGC, CMS algorithms and their interactions with MRJ, noting how GC behavior changes with the characteristics of the application, input data size, JVM heap size, and the number of threads. The main methodological contribution of the paper is the introduction of memory management auto-tuning techniques based on ML, particularly Random Forest Classifier. These techniques aim to dynamically select the most appropriate GC policy (they define the policy based on six different combinations of GC type flags and young/old generation ratio flag values) for a given MRJ application scenario, considering factors like benchmark-specific behavior, heap size requirements, and the number of worker threads. Their results show that the auto-tuning system achieves 10% of better MRJ performance in 75% of the benchmark tests. As a result, they present a comprehensive study on the scalability of the MRJ framework, and the impact of Java GC on its performance, and propose an innovative auto-tuning solution that uses ML to improve GC efficiency. This results in substantial performance gains in various benchmark tests, demonstrating the feasibility and benefits of applying auto-tuning GC techniques in multi-core Java MapReduce environments.

2.4.7 JVM Configuration Management and Its Performance Impact for Big Data Applications

Sahin *et al.* [29] explore the relationship between JVM configurations, JVM options for heap, GC parameters, and the performance of big data applications. Without addressing the auto-tuning problem, the authors only investigate how tuning these JVM parameters affects application performance without a change in hypervisors, OS, or JVM. They provide an extensive measurement study focusing on JVM heap size, young-old generation ratio, and various GC algorithms, including SerialGC, ParallelGC, and the CMS collector. The study reveals the following key observations: increasing the JVM heap size beyond a certain point does not always improve performance, heap space errors can occur even when the heap is not filled, and these errors can often be resolved by adjusting the options like initial heap size, young-old generation ratio without increasing the heap size. Furthermore, it shows that applications can achieve comparable performance with smaller JVM heap sizes through careful tuning of the heap structure and GC parameters. These findings suggest that software developers can optimize big data application performance through better JVM configuration management, achieving efficient big data computing without extensive system modifications. The paper concludes that adjusting JVM configurations is a feasible approach to improve the performance of big data applications.

2.4.8 CGPTuner: a Contextual Gaussian Process Bandit Approach for the Automatic Tuning of IT Configurations Under Varying Workload Conditions

Doni *et al.* [30] introduce a novel approach to automatically tune the configuration of IT systems, particularly Cassandra and MongoDB database management systems, across different layers of an IT stack including JVM, OS and physical machine. Their method, CGPTuner, is based on an ML optimization algorithm called Contextual Gaussian Process Bandit Optimization (CGPBO). CGPBO (based on Bayesian optimization with a contextual extension) primarily focuses on tuning configurations dynamically. The study addresses challenges presented by changing software versions and parameter effects under different workloads. The CGPTuner adapts the configurations of the database management systems, JVM, and OS layers to optimize system performance. It analyzes performance metrics such

as response time, throughput, and memory usage to determine the most effective settings under current conditions. The authors test the CGPTuner using Cassandra and MongoDB database management systems, by simulating different workload patterns. The results show that CGPTuner can effectively adapt its tuning strategies to match the workload variations, optimizing configurations within 30 tuning iterations. The CGPTuner is shown to reduce response time and adjust memory requirements as necessary, depending on the workload, proving its efficiency and adaptability without the need for a historical knowledge base.

2.4.9 Optimization of Java Virtual Machine Flags using Feature Model and Genetic Algorithm

Canales *et al.* [31] explore a novel approach for tuning JVM options using genetic algorithms (GA) and a feature model. This research focuses on optimizing over 500 JVM flags based on the flag hierarchy in the study 2.4.1. They aim to tune the flags related to compiler behaviors, memory management, and GC processes. The authors introduce a feature model to handle the incompatibilities among these flags effectively, for instance, some flags might not be relevant for some configurations. This feature model is the input for the GA, which aims to optimize JVM configurations by evolving flag settings across generations to improve program execution times. GA is a computational method inspired by natural selection with genes, individuals, and generations; used to model flags, where genes represent flag values, individuals are sets of flags that are selected based on performance impact on the JVM where the objective function measures it as the execution time of the benchmark that runs with the provided set of flags, extended by control genes to constraint their model. Poor performers are discarded, while successful configurations produce, mutate, and pass into subsequent generations. They use the DaCapo benchmark suite to evaluate the effectiveness of their approach. The results show significant performance improvements that reach 23%, with optimized configurations outperforming default settings. This validation confirms the potential of combining feature models and genetic algorithms for JVM flag optimization by reducing manual tuning efforts and providing more efficient execution of Java programs.

2.4.10 Summary

JVM tuning for better performance is a difficult task, as we see from previous work done in this field to find solutions or to analyze the effects of different JVM options on performance. To conclude this section, we discuss the similarities and differences between our project and the related work below:

- Tuning options: While auto-tuning-related studies focus mainly on tuning the heap size, GC related options, and the ratio between young and old generations, we opt to tune the upper and lower boundaries of the young generation usage within a given heap size.

- Number of runs: Studies for auto-tuning are based on running the application many times to reach the optimal performance parameters. Our project takes a different approach. It aims to identify the best flag values for optimal performance by analyzing the current behavior of a Java application during runtime using information from a GC log of the particular Java application executed once.

- Optimization technique: The studies utilize various optimization algorithms such as Bayesian optimization, ParamILS, and different regression techniques. Although they show performance improvements with these techniques, we utilize different models considering the nature of our dataset. Since they are exploring the search space of different configurations of JVM options with Bayesian optimization whereas we extract information from the GC logs and train the ML models. However, since we need to tune the hyperparameters of these models, we use Bayes search for this task.

- Benchmark selection: Some studies utilize DaCapo and SPEC benchmarks, which we have also chosen for our experiments.

- Garbage collector: Various garbage collection algorithms are considered for different studies. Because G1 is the default in the JDK version we use (see 4.1.1) and as the study [10] concludes that G1 collector is the most efficient, we consider only G1 in our project.

Chapter 3

Design and Implementation

This chapter provides details on the steps we follow during the project, JVM flags we use and auto-tune, pipeline design (including feature engineering, model training, mapping the predicted values to JVM flags), and explaining experiments to measure performance improvements.

3.1 Project Workflow

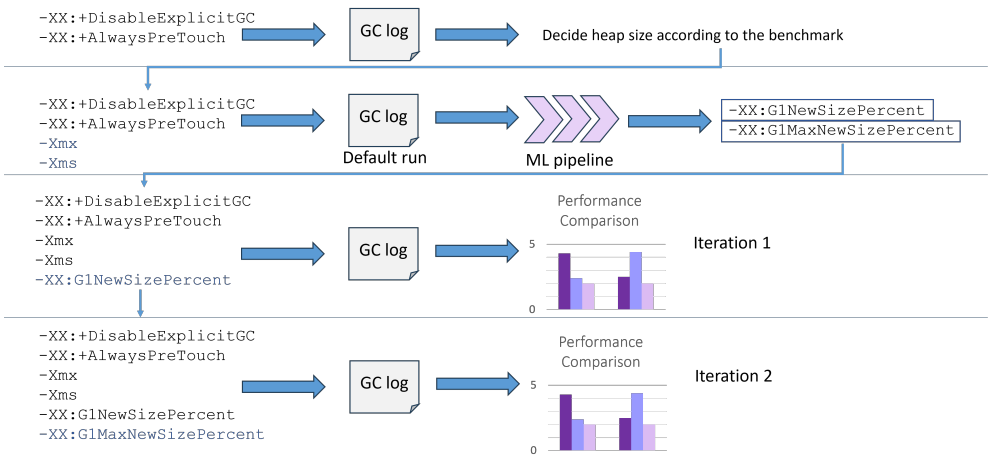


Figure 3.1: Project workflow: The project workflow: Heap size selection, ML pipeline utilization and performance measurements. Each line represents one execution of the application. On the left, we see the flags used in that execution.

Figure 3.1 shows the workflow of this project. The steps shown in this diagram are considered for only one benchmark or Java application at a time.

Each row in the diagram represents one run of the application. We utilize two flags in all of the runs that are given below with the reasonings why we use them:

- `-XX:+DisableExplicitGC`: This flag [8] disables the explicit GC invocations performed by calling System.gc().
- `-XX:+AlwaysPreTouch`: With this flag [8], JVM allocates all the physical memory pages for the entire heap beforehand instead of doing it incrementally as the application runs. We use this flag to minimize the JVM fluctuations during its warm-up phase and to prevent any effect of the OS on the performance measurements, thereby ensuring its integrity and accuracy.

The first execution is a necessary step for us to decide the heap size of the benchmark. Therefore, we do not set any other flags besides `-XX:+DisableExplicitGC` and `-XX:+AlwaysPreTouch`. The selection method of the optimal heap size varies depending on the benchmark being used. The Specjbb2015 benchmark operates dynamically, without a fixed workload. It evaluates a system's performance by incrementally increasing the number of requests it must handle until the system reaches its limit. Consequently, the benchmark yields higher throughput and latency values with larger available resources. To ensure a more accurate representation, we advocate for a JVM configuration that maximizes memory allocation. Therefore, we selected a memory allocation of 25 GB, as the maximum capacity of the machine we use is 32 GB. Remaining 7 GB ensures that sufficient resources remain available for background tasks, preventing undue strain. For the DaCapo/h2 and DaCapo/cassandra benchmarks, we used a similar approach as suggested by Li *et al.* [32] for optimal heap size selection. From the default GC log, we observe the maximum size of the live set after the GC events and set the heap size for these benchmarks to three times larger than this value to ensure the stability of the benchmark.

Once we select the optimal heap size, we set the `-Xmx` and `-Xms` flags, which determine the maximum and minimum (or initial) heap size in bytes, in the second run. We consider this run as the "default run" and record the performance of this run. We will be comparing our tuned performances with the performance of this run in Chapter 4. The GC log of this run is the input of our ML pipeline to get the suggested values of the two flags we want to auto-tune.

The last two rows of the diagram show the tuned runs – there are five tuned runs where we use the suggested flag values by five different ML

models. With the first iteration, we aim to improve the JVM for throughput and the second iteration is for optimizing the latency compared to the first iteration while not impacting the improved throughput much. We expect latency to degrade when the throughput improves after the first iteration and aim to bring it to a better level. Therefore, we begin by setting the suggested value for `-XX:G1NewSizePercent` flag and comparing the performances of the tuned runs with the default run, first iteration. In the last row, we set `-XX:G1MaxNewSizePercent` flag together with the previous configuration and compare the performances as the second iteration.

3.2 Pipeline Design

The input to our ML pipeline is the GC log of a Java application. As explained in Section 2.2, the pipeline outputs two flag values, namely `-XX:G1NewSizePercent` and `-XX:G1MaxNewSizePercent`, which serve to optimize throughput and latency, respectively.

This section explains the pipeline in five sections: feature engineering, selection of features, selection of sample data after model training to predict suggestions, hyper-parameter tuning, and mappings of suggested values to flag values for tuning.

3.2.1 Feature Engineering

Feature engineering is the first step in the pipeline after parsing the GC log. By parsing the log file, we extract various features that are listed in Section 2.1.3 by using regular expressions, an example can be found in Appendix A.1.2. Once the parsing is complete, we convert all timestamps into seconds, beginning at second zero and continuing to the end. The conversion facilitates performing our calculations in the pipeline. In addition to these features, we calculate an allocation rate to understand the memory utilization pattern. Moreover, we want to add new features to the data that are related to the performance of the overall application by calculating some performance (throughput and latency) related metrics like net application time, maximum pause time, and GC frequency. To obtain these metrics, our approach is to slide a time window over the data and create new processed data consisting of averaged values of the features and the calculated performance related metrics within that window. We explain these steps to pre-process the data in this subsection along with filtering the features at the end of feature engineering step.

Allocation rate

The primary objective of this project is to gain insights into the behavior of the application during runtime by analyzing its GC log. The allocation rate, which can be determined using the formula presented in equation 3.1, is a key metric that offers valuable information on the memory utilization patterns and the behavior of JVM during the execution of an application, which can be considered as the data allocation speed of the application.

$$\text{Allocation rate} = \frac{(\text{Heap usage before GC at } t_2 - \text{Heap usage after GC at } t_1)}{t_2 \text{ - } t_1} \tag{3.1}$$

It is defined as the allocation between two consecutive GC events over the time between them. To explain the equation 3.1, t_2 and t_1 indicate the times when two consecutive GC events occur. We calculate the difference between the heap usage before the latter GC at time t_2 and heap usage after the first GC at time t_1 and divide by the time difference. This is an important metric because the heap gets occupied faster with a higher allocation rate which triggers more GC. As such, it is an essential metric for understanding and optimizing the performance of the application.

One-hot encoding for categorical features

As we mentioned earlier in Section 2.1.1, the presence of the distinct pause types can help understand how different garbage collection activities impact memory usage patterns. As these pause types represent categorical data, they need to be transformed into a one-hot encoding representation, where each category is represented as a new column or feature in the dataset, and boolean (true or false) values are assigned to the corresponding rows. For example, the extracted features contain a feature with name ”Pause Type” where the values of this features are ”Normal”, ”Concurrent start”, ”Mixed” and ”Prepare mixed”. With one-hot encoding, we convert these values as features in columns and set them as ”True” or ”False”, an example view is given after applying one-hot encoding in Appendix A.1.4.

This approach facilitates the analysis and prediction phase of memory usage patterns for ML models in Java applications that use G1 garbage collectors.

Conversion of units

When analyzing the GC log, we can extract specific units of measurement such as kilobytes or megabytes for memory-related information and microseconds or milliseconds for GC pauses. To improve the accuracy of our models, we convert the units of related features to have a common standard among the units. For example, in our Java benchmarks, the units of all memory-related features appear in the GC log as megabytes except for the metaspace (the concept of metaspace explained in Section 2.1.3), which is in kilobytes. We convert them into megabytes to keep memory-related features in the same unit.

Sliding time window

We use a time window approach to process our data, meaning that a window with a defined size goes through the extracted features from the GC log by capturing different timestamps depending on the window size and sliding amount. In each window, we compute the average of the numerical features and fill the boolean feature values with the predominant boolean value within that window. For example, if there are more ”True”s than ”False”s, we fill with the ”True” value. Moreover, performance metrics such as maximum GC pause time, total GC pause time, net time spent on application threads (see formula 3.2), and GC frequency are calculated for each window locally and added to the data.

We consider timestamps extracted from the log file to decide the time window size. The difference between timestamps is not equal to each other because the GC pauses take different amounts of time, which means that sometimes the difference can be quite large relative to the execution time of the application. Therefore, the window size calculation is generic to the applications, equal to the maximum time difference between two consecutive timestamps. As a design choice, the sliding amount of the windows is equal to 20% of the window size.

The underlying concept of our approach is to compute the defined performance-related metrics for each window, by obtaining information on how a particular Java program behaves under different workload and memory utilization conditions. We believe that this approach allows the models to collect insights from these smaller-scale applications, thereby facilitating more accurate predictions for the entire application. In other words, what we investigate in this research is to what extent the ML model can learn the relation between the features we can obtain from GC logs and how they might affect the overall application performance. Therefore, these metrics

are computed for each time window. We conduct these calculations in the following manner:

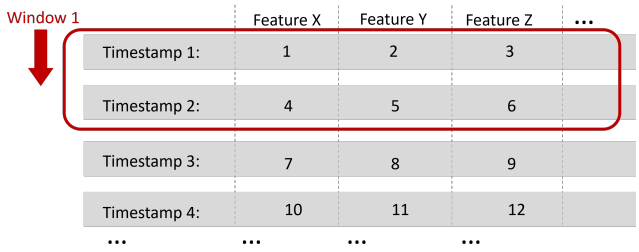
- Defining a metric related to the throughput of an application can be done by measuring the amount of work performed by the application within a specific time frame. This metric can provide valuable insights into the performance and efficiency of the application. In other words, we should exclude the total time spent for GC pauses within that window from the window size. We do not consider the effects of concurrent pauses as mentioned in Section 2.1.1. This metric, net application time, can be expressed as

$$\text{Net time} = \frac{(\text{time window size} - \text{total GC pause time in window})}{\text{time window size}} \times 100 \tag{3.2}$$

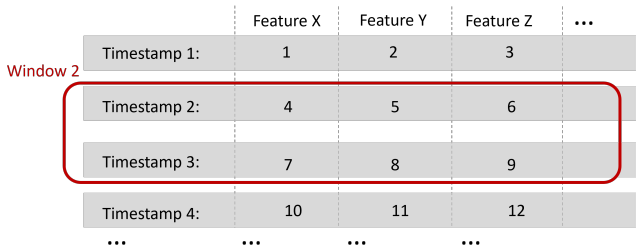
- Total and maximum GC pause time within a window
- The number of GC events occurred within a window — GC frequency.

To explain the sliding time window approach better at an abstract level, Figures 3.2 and 3.3 can be considered. They are a simple demonstration of this approach. Figure 3.2a provides a visual representation of a simplified representation of the data we obtained from a GC log. Each row contains timestamps and corresponding features. Features X, Y, Z with dummy numbers represent the features we extract from the GC log as we listed in Section 2.1.3. We define a window size in time units. Then, we calculate the performance metrics such as the net execution time spent for the application, the GC frequency, total and maximum time spent for GC events within this window (GC pause times), get the averages of the values for other numerical features like heap size and create a new processed dataset with them.

Figure 3.3 indicates an example of processed data after these computations where each row corresponds to a window and includes performance metrics. After completing the first window, we slide it by the defined amount as shown in Figure 3.2b where this amount may contain different numbers of timestamps (since they are not equally distant) and repeat this process for the remaining windows until we reach the end of the file. A real example of how our data looks after each analysis step (initially GC log, extracted features, and processed data in tabular format) can be found in Appendix A.1.



(a) Representation of the first time window: Sliding time window demonstration with dummy values – First time window



(b) Representation of the time window after sliding once: Sliding time window demonstration with dummy values – After sliding the window once

Figure 3.2: Representation of the time window after sliding once: Simple demonstration of the time window approach to pre-process the data. Each column represents the features that we have extracted from the GC logs by parsing them where the very left column is the timestamp of the events given in the logs. We use a time window with the size of the maximum time difference between these timestamps, process each window data to create a new dataset, and slide it until the end of the log file.

Feature selection and train/test partitioning

Up to this point, we have created a new data set by incorporating existing features and augmenting it with additional ones, such as new engineered features for each window. We aim to recommend the young generation size, comprising the combined area of Eden and the survivor regions. Notably, the Eden size significantly outweighs the survivor size. Therefore, as a design choice, the output label is set to Eden usage before GC events. We choose to use the predicted Eden utilization as an input to calculate the recommended young generation size for the two JVM flags we are tuning.

The next step is obtaining the correlation between the features and the label

	Performance metric	Feature X	Feature Y	...
Window 1:	10	4	3	
Window 2:	20	3	7	
Window 3:	30	9	8	
Window 4:	40	13	12	
...	...	...	...	

Figure 3.3: Representation of the dataset after processing with time window approach: Processed data demonstration after applying sliding window approach. Each line represents the information for each time window with dummy numbers. This new dataset contains the calculated performance metrics for the time windows and processed numerical and boolean features.

and analyzing their prediction powers (the more they correlate, the more they have prediction power). Then, we keep the features that relate more with the output label in the processed data since it can allow the model to understand the relation among them.

Table 3.1 shows an example of the correlation values between the features and the label. The features we select should be helpful to the ML models to capture the relations between them and the label they predict. Therefore, we choose the features with the correlation values larger than 0.4 and less than -0.4. The range (-1.0, 1.0) indicate perfectly negative and positive correlations, respectively. The stronger their correlation, the greater their predictive capability. After selecting the features, we use the train/test split function from Scikit-learn [33] with 20% ratio that splits the dataset into 80% train and the remainder is the test set randomly.

3.2.2 ML models

As mentioned in Section 2.3, we use SVR, GB, RFR, ET and GPR in this project. We train these models with the processed GC log data that we obtained after the feature engineering step in the pipeline to predict optimal values for `-XX:G1NewSizePercent` and `-XX:G1MaxNewSizePercent` flags we want to tune for better performance of the Java application.

For the given dataset, we also need to find the hyperparameter values for each model that result in the lowest mean squared error [34] before training our models. As explained in Section 2.3, Bayes search method (BayesSearchCV) from sk-learn library is conducted for hyperparameter value selection.

Table 3.1: The correlation values between the features and the label: The correlations between the features from processed data (processed data of SPECjbb2015 default run GC log) and the label (Eden usage before GC) of ML models. The correlation values are within the range -1 to 1 where they represent negative and positive fully correlations, respectively. Since heap size is a fixed number which we set already, there is no correlation between heap size and the label (NaN).

Features	**Correlations with the label** (Eden usage before GC event)
Eden usage before GC	1.000000
Eden size	0.993103
Survivor size	0.977801
Metaspace usage after GC	0.973103
Metaspace usage before GC	0.973103
Metaspace size after GC	0.972862
Metaspace size before GC	0.972862
Heap usage before GC	0.924420
Allocation rate	0.853675
Old generation usage before GC	0.749305
Old generation usage after GC	0.732296
GC frequency	0.605036
Total pause time	0.576596
Heap usage after	0.534419
Time window start second	0.391966
Survivor usage before GC	-0.074918
Survivor usage after GC	-0.183772
Pause type: Normal	-0.417891
90th percentile of pause times	-0.489509
50th percentile of pause times	-0.517831
Pause type: Mixed	-0.527726
Pause type: Prepare mixed	-0.527726
Pause type: Concurrent start	-0.545733
Net application time	-0.576596
Maximum pause time	-0.627146
Heap size	NaN

3.2.3 Inference Data for Predictions

For the suggestion phase, it is crucial to inform the models about the objectives of the predictions. This facilitates their understanding of which aspects of

application performance require improvement. Our methodology involves selecting rows with the poorest values of local performance-related features for targeted improvement (net application time for throughput, maximum GC pause times for latency). We first aim to maximize throughput by improving the local net execution time spent for the application (this is directly correlated with the local GC frequencies). Secondly, optimizing latency by improving the local maximum pause times to optimize the GC pause times of the overall application without significantly impacting the optimized throughput. We choose the inference data for each objective as the following, respectively:

- Throughput: For controlling the GC frequencies, which is a proxy for throughput, we prepare our data for inference by gathering the several rows with the smallest values of the net execution time for application threads and set these values to the maximum possible value of this metric so that the model can predict the Eden usage values that maximize the application throughput.
- Latency: For controlling the time spent for each GC event, which is a proxy for responsiveness, a pause time threshold is defined, by default 200 milliseconds in G1, trying to meet this requirement as a soft target. We pick the rows with maximum pause time values for each window that are greater than the defined threshold, and these values are set to the threshold value so that the model can provide Eden usage values that minimize the maximum pause time of the overall application to improve the responsiveness or latency of the application.

3.2.4 Model Predictions and Mapping to Flag Values

The label for ML models is established as the Eden usage before GC events, indicating that the models provide predictions for this metric. Given that there is no direct correspondence of this value with the JVM flags, we have defined straightforward mapping equations to derive the results.

$$\text{Min percent} = \frac{\text{min(predicted Eden usage + Survivors usage)}}{\text{maximum heap size}} \times 100 \quad (3.3)$$

$$\text{Max percent} = \frac{\text{max(predicted Eden usage + Survivors usage)}}{\text{maximum heap size}} \times 100 \quad (3.4)$$

These mapping formulations are given in equations 3.3 and 3.4 for `-XX:G1NewSizePercent` and `-XX:G1MaxNewSizePercent` flags, respectively. We select multiple rows from the data for this prediction as mentioned in the previous subsection. This results in multiple rows of predictions. We sum the size of Survivor regions usage before GC events and the predicted Eden usage amount before GC events from the selected rows and get the minimum or maximum among the multiple rows depending on which flag we are mapping to, and divide this value by the maximum heap size to get the ratio of this amount of usage within the given heap size.

Chapter 4

Evaluation and Results

This chapter first describes our experimental configuration and our methods for the evaluations. Then, we explain our findings from the three benchmarks and compare the performance of the ML models. We finalize this chapter with the discussion.

4.1 Experimental Configuration

After deciding the heap size and obtaining the suggested flag values, we set these flag values to tune the JVM. For each benchmark, the tests are carried out in two iterations as shown in Figure 3.1. Our ML pipeline recommends values for the flags to optimize the throughput while achieving tolerable latency of the application by using the GC log of the default run. We first set the recommended value for the `-XX:G1NewSizePercent` flag in the first iteration to improve the throughput. This iteration may reduce the responsiveness of the application. Therefore, we aim to achieve a tolerable latency by setting the recommended value for the `-XX:G1MaxNewSizePercent` flag as an additional configuration to the first iteration to control the latency in the second iteration.

- First iteration: We configure the `-XX:G1NewSizePercent` flag (minimum percentage for young generation usage) using the suggested value by the ML pipeline. We expect overall throughput-related metrics to be better than the default run together with less frequent GC events. In this step, we might see a slight increase in the maximum GC pause times of the time windows, which serve as an indicator of program latency.

- Second iteration: To control the maximum pause times or latency, we

also set `-XX:G1MaxNewSizePercent` flag (maximum percentage) along with the same configuration in the previous iteration. We expect to have improved latency metrics compared to the first iteration while minimally impacting throughput.

4.1.1 Hardware/Software Setup

There are several tools used to conduct this project. The necessary software can be listed as Python libraries like sklearn, matplotlib, pandas, numpy, and re. We use two identical Oracle servers with Oracle Linux OS, twelve cores of 2.7 GHz Ivy Bridge-EP [Xeon E5-2697 v2] and memory of 32 GB. We use Oracle's HotSpot Java Runtime Environment (JRE) version 11.0.22 to generate GC logs from benchmarks and collect performance measures, the SPECJbb2015 benchmark [35] and the DaCapo benchmark suite [36] and GitHub for version control.

4.1.2 Reliability and Validity of the Results

The experimental setup involved running the applications on two Oracle servers with identical hardware specifications. The servers were dedicated exclusively to the experiment and we do not run any other processes actively during testing. However, we may observe deviations in the experiment results due to external factors like network conditions or hardware issues, or background processes that the OS itself is running like system daemons or kernel processes. Nonetheless, this setup allows us to have a reliable enough performance comparison between the application run with default settings and the tuned run with the recommended JVM flag values since they are identical machines and we validate our results by running the tests multiple times. To ensure the validity of performance measurements for applications, it is advisable to rely on benchmark performance results. These results are known to be a credible source of measurement and can be trusted in assessing the performance of an application.

4.2 Evaluation Method

The Java benchmarks used in this research are SPECjbb2015 and DaCapo (cassandra, h2). The explanations of these benchmarks are given in Table 4.1. The quality observation of performance improvement for these benchmarks is based on the performance metrics they provide. SPECjbb2015 reports a Java

Operations Per Second (jOPS) metric called max-jOPS. According to its user guide [37], it represents the throughput of the benchmark run.

Table 4.1: Java benchmarks: Descriptions of the benchmarks used in this project: SPECjbb2015 [35] and DaCapo [36]

Benchmark name	**Description**
SPECjbb2015	This benchmark is simply a Java client/server application simulating sales/customer interactions, data management and business intelligence related operations.
DaCapo/cassandra	It simulates an open-source and NoSQL database management system, Apache Cassandra, used to manage massive amounts of data.
DaCapo/h2	It simulates H2 database engine which is an open-source and in-memory database.

On the other hand, cassandra, and h2 benchmarks from DaCapo report only latency measurements including simple and metered latencies in various percentiles. As Zhao *et al.* addressed in their paper [38], request latency is influenced by three main factors: uninterrupted computation time, interruptions such as scheduling and garbage collection, and request queuing time. DaCapo's metered measurement method accounts for all three factors, including request queuing, essential for cloud-based services where requests arrive at servers at a metered rate. Hence, in this project, we consider metered latency measurements. We decide to observe the 50th percentile (P50) and 99th percentile (P99). The 50th percentile latency denotes the median latency, where half of the requests experience a latency shorter than this value, and the other half experience a longer latency. On the other hand, the 99th percentile latency (also tail latency) represents the point below which 99% of the requests fall, indicating that 99% of the requests experience a latency shorter than this value, while 1% experience longer latency. These metrics serve as crucial indicators of system performance under load, with shorter latencies implying quicker response times and better overall efficiency in handling requests.

Additionally, it is worth considering that the execution time of DaCapo benchmarks correlates with their throughput since the same amount of work is done in each run of the applications, a similar approach used in the paper [27]. On the contrary, this is not the case for SPECjbb2015 because it tries to achieve a certain amount of time in each run. We observe max-jOPS to monitor the change in throughput. Moreover, decreasing the total pause time

of the application would be another indicator of better performance.

The decrease or increase may be interpreted differently for performance measurements. Only the metric reported by SPECjbb2015, max-jOPS, is the higher the better since it represents the amount of work the application can achieve. However, shorter execution times (only for DaCapo) and smaller latency values for tuned runs indicate performance improvement.

4.3 Results

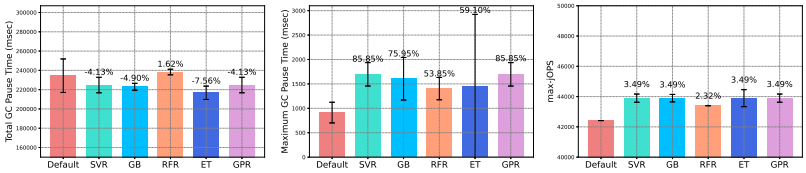
As mentioned in Section 4.1, we conduct the experiments in two iterations. The pipeline suggests the value of two JVM flags for five different ML models. We set `-XX:G1NewSizePercent` and `-XX:G1MaxNewSizePercent` (minimum and maximum percentage of the size of the young generation in heap size, respectively), only the minimum percentage flag in the first iteration of the experiments, and both the minimum and maximum percentage flags in the second iteration. In other words, we first aim to improve throughput by reducing GC frequency followed by an attempt to restrain the latency by reducing the maximum GC pause times in the second iteration. The suggested values by models for the flags we are tuning are given in Appendix A.3.

This section provides our findings and results from the experiments. 4.3.1 shows the results from both iterations. We compare the performance of each ML model on performance changes between default and tuned version in 4.3.2.

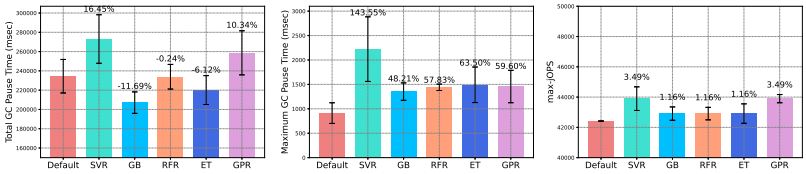
4.3.1 Comparison of the Default and Tuned Runs of the Benchmarks

We present the results from SPECjbb2015 and DaCapo benchmarks in this section. All bar graphs indicate comparisons between the application performance of default runs and five tuned runs based on the suggestions of five different ML models. The y-axis shows the performance measurement, the x-axis has six bars starting from the default run on the very left followed by the ML models, SVR, GB, RFR, ET and GPR. Each bar is the median of multiple runs of the same configuration. The black line indicates the standard deviation of these runs. The change between the default and each tuned run is shown in percentage on top of the corresponding bar.

SPECjbb2015:



(a) Results of iteration 1: Optimizing for throughput, only `-XX:G1NewSizePercent` (minimum percentage flag) is set in tuned runs.



(b) Results of iteration 2: Optimizing for latency without impacting improved throughput, both `-XX:G1NewSizePercent` and `-XX:G1MaxNewSizePercent` (minimum and maximum percentage flags) are set in the tuned runs.

Figure 4.1: Results of SPECjbb2015 benchmark: Comparison graphs of the default and tuned runs for total GC pause time, maximum GC pause time of the application and reported throughput metric max-jOPS by SPECjbb2015 benchmark. The y-axis indicates the performance measurement (total pause time, maximum pause time and max-jOPS from left to right) and the x-axis shows the results from the default run on the left followed by the results of each model that are SVR, GB, RFR, ET and GPR from left to right. Each bar is the median of 5 runs of the same configuration, the black lines represent the median ± the standard deviation of these runs, and the percentages show the change compared to the default result of the related tuned version.

From Figures 4.1a and 4.1b, we observe the results of the SPECjbb2015 benchmark for the first and second iterations of the tuned runs. The graphs on the left are the results of the total time spent for all the GC events during the execution of the application – total GC pause time, we see the results of maximum pause time of the programs in the center, that is, the maximum amount of time spent for the GC events and the graphs on the right show the max-jOPS metric reported by SPECjbb2015 benchmark. Total GC pause time and max-jOPS can be considered as throughput-related measurements where total pause time should be lower and max-jOPS should be higher value for better throughput performance. On the other hand,

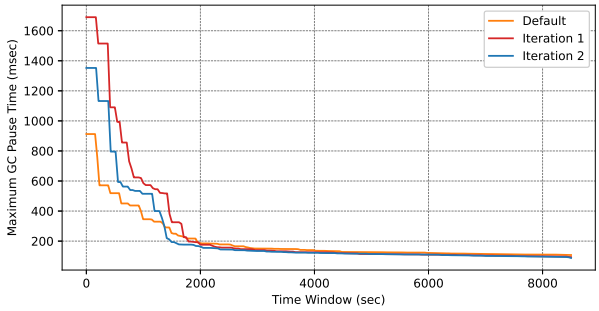
maximum GC pause time of the overall program gives us insights about the responsiveness of the program. As we mentioned in Section 4.1, we aim to achieve tolerable responsiveness or latency without impacting the improved throughput significantly with the second iteration. Therefore, it is worth to note maximum GC pause time of the program. To dive deeper, we first focus on the total pause time in both iterations followed by max-jOPS. Then, we take a look at maximum pause time of the overall application and the maximum pause times in the time windows since we try to control them locally in time windows.

We see up to 7.56% shorter total pause time with model ET in the first iteration in Figure 4.1a. Except for the result of RFR, total pause time is slightly improved in this iteration.

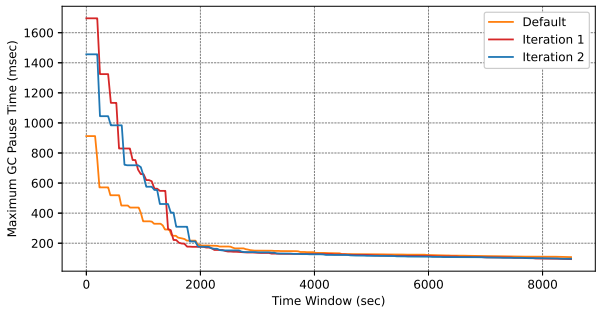
In the second iteration, the throughput is expected to drop slightly and the latency (maximum GC pause times in time windows) should improve compared to the first iteration. We observe that the results for SVR and GPR are negatively affected; however, GB shows a further improvement from 4.9% to 11.69%. When we pay attention to max-jOPS results, we see more statistically separable results from the default run. It is improved by 3.49% for all models except for RFR in the first iteration whereas this number is dropped for GB and ET in the second iteration.

In the maximum pause time graphs, we observe a drop in the second iteration in GB and GPR results as expected. Moreover, the variability of the maximum pause times is reduced in general except for SVR. This means that we can reduce the peak pause time values that is spent for GC events and increase the reliability by reducing the deviations in different runs.

As mentioned in Section 3.2.1, our approach is to improve the performance-related metrics we calculated for each time window in the data to improve overall performance. We try to control the GC frequency with the `-XX:G1NewSizePercent` flag in iteration 1, which may cause a longer maximum pause time in time windows since we are increasing the size of the young generation. In other words, the minor collection time is directly proportional to the size of the young generation. However, the `-XX:G1MaxNewSizePercent` flag we set in the second iteration can help us limit the maximum pause times. Eventually, our objective is to improve throughput while achieving acceptable latency. Figure 4.2 helps us observe the change in latency of the GB and GPR, respectively in Figures 4.2a and 4.2b, i.e., to observe the change in maximum GC pause times in each time window as the proxy of latency. Each line graph is the median of maximum pause times in local time windows from multiple runs of the same configuration. The y-



(a) The average Maximum Pause Times vs. Time Windows of the tuned runs of SPECjbb2015 that are configured with the suggested flag values by GB model



(b) The average Maximum Pause Times vs. Time Windows of the tuned runs of SPECjbb2015 that are configured with the suggested flag values by GPR model

Figure 4.2: The graphs of maximum GC pause times in local time windows for SPECjbb2015: Maximum GC Pause Times vs. Time Window graphs from tuned runs with the suggested flag values by GB and GPR models. The line graphs show the average maximum GC pause times in local time windows of multiple same configured runs where the red and blue lines represent iteration 1 and 2, respectively, and the orange is from the default run.

axis represents the maximum GC pause time for each time window where the corresponding time window is shown on the x-axis. We slide the windows from the beginning till the end of the GC log of a particular run.

Both GB and GPR are interesting examples because both meet our expectations, that is, optimizing the throughput in the first iteration and improving latency in the second iteration compared to the first one without sacrificing from the improved throughput much. The results from both models achieve better responsiveness in the second iteration unlike other models,

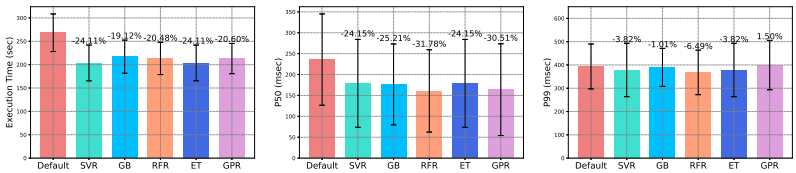
although the maximum pause times in the second iteration are still worse than the default, which still meets the expectation since the throughput is improved in the second iteration compared to the default. The max-jOPS of GB drops slightly in the second iteration which is still better than the default run whereas GPR is consistent with 3.49% better max-jOPS.

For both cases, we observe that the average maximum pause times for time windows decrease in the second tuning iteration shown with red (iteration 1) and blue (iteration 2) lines. To clarify more, the top 5 maximum pause times of the overall applications (the median of five runs of the same configurations) are shown in Table 4.2. Although the maximum pause time of the second iteration is still higher than the default, these models achieve better throughput without sacrificing too much latency. The five largest pause times of other tuned runs are given in Appendix A.5.

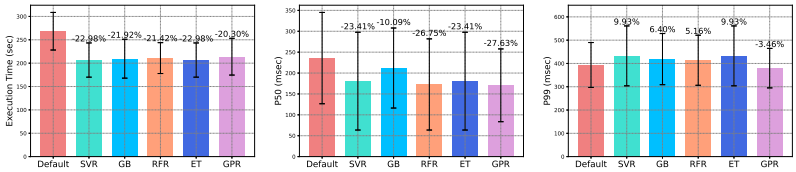
Table 4.2: The five highest maximum GC pause times: The maximum GC pause times of the default run and the tuned runs of SPECjbb2015 benchmark where tuned runs are executed with the suggested flag values by GB and GPR models. The medians of the top 5 maximum pause times of the overall programs of the same configurations are shown. (iteration 1 depicts the scenario where only the minimum percentage of young generation usage flag is set, while iteration 2 illustrates the situation where both maximum and minimum percentage flags are set concurrently).

	Default	GB		GPR	
		iter 1	iter 2	iter 1	iter 2
Top five largest pause times (msec)	912.42	1690.07	1352.33	1695.69	1456.24
	758.12	1514.76	1132.24	1324.57	1044.80
	571.23	1090.34	795.72	1133.38	984.08
	518.96	855.97	562.82	829.85	718.30
	450.58	623.84	539.82	688.96	705.99

DaCapo/cassandra:



(a) Results of iteration 1: Optimizing for throughput, only `-XX:G1NewSizePercent` (minimum percentage flag) is set in tuned runs.



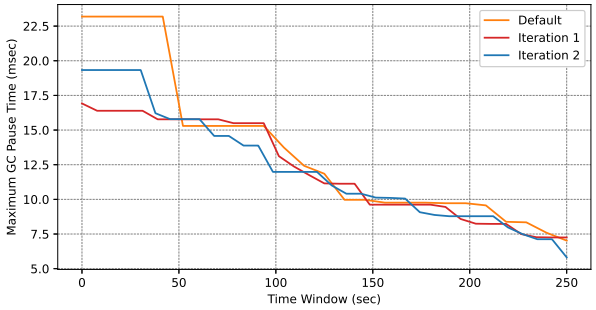
(b) Results of iteration 2: Optimizing for latency without impacting improved throughput, both `-XX:G1NewSizePercent` and `-XX:G1MaxNewSizePercent` (minimum and maximum percentage flags) are set in tuned runs.

Figure 4.3: Results of DaCapo/cassandra benchmark: Comparison graphs of the default and tuned runs for program execution time and reported latency metrics 50th (P50) and 99th (P99) percentiles of the latency by DaCapo/cassandra benchmark. The y-axis indicates the performance measurement (execution time, 50th and 99th percentile of the latency from left to right) and the x-axis shows the results from the default run on the left followed by the results of each model that are SVR, GB, RFR, ET and GPR from left to right. Each bar is the median of 20 runs of the same configuration, the black lines represent the median ± the standard deviation of these runs, and the percentages show the change compared to the default result of the related tuned version.

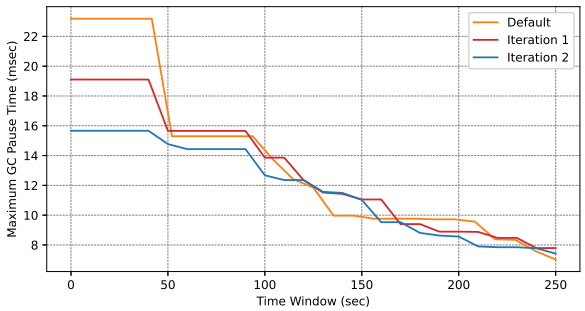
As explained in Section 4.3.1, we monitor the execution time of the application as a proxy of the throughput of the program and 50th and 99th percentile latency metrics (P50 and P99) reported by DaCapo benchmark. Figure 4.3 indicates these metrics measured from the default and tuned runs of the DaCapo/cassandra benchmark.

If we focus on the graphs given in 4.3a and 4.3b, we observe that setting `-XX:G1MaxNewSizePercent` flag (second iteration) does not have an improvement except for GPR model results in P99 graphs. By tuning this flag, we try to minimize the maximum time spent for GC events in each local time

window within our GC log data. This benchmark already achieves GC pauses within a short amount of time (up to 22 milliseconds) since it simulates a real-world scenario of Apache-Cassandra database workload, which is already designed to handle large amounts of data with low-latency reads and writes [39]. Nevertheless, the application can complete its fixed number of tasks within a shorter time which means the throughput is improved.



(a) The average Maximum Pause Times vs. Time Windows of the tuned runs of DaCapo/cassandra that are configured with the suggested flag values by GB model



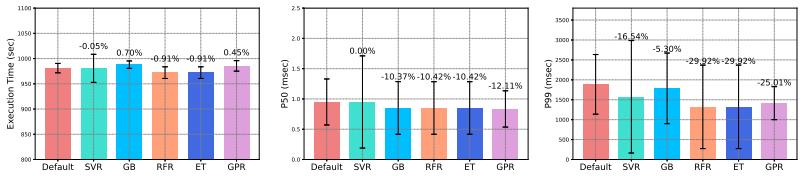
(b) The average Maximum Pause Times vs. Time Windows of the tuned runs of DaCapo/cassandra that are configured with the suggested flag values by GPR model

Figure 4.4: The graphs of maximum GC pause times in local time windows for DaCapo/cassandra: The average maximum pause times of the local time windows of DaCapo/cassandra benchmark. Each line represents the average of all runs of the same configuration. The orange line represents the mean maximum pause times in time windows of the default run. The red line is the mean maximum pause times in time windows in iteration 1 and the blue line is the result of the second iteration of the tuning with the flag values suggested by the GB and GPR model.

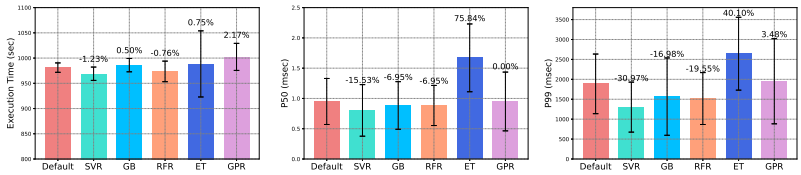
We can now focus on the local time windows of the run executed with GPR recommendations to see whether maximum pause times confirm this behavior and GB as an example of a case that does not meet our expectations. Figure 4.4 shows the median of the maximum pause times of all 20 runs configured with values from GB and GPR models, Figure 4.4a and 4.4b, respectively. In GPR results, we observe a similar trend in the maximum pause times of each local time window as in SPECjbb2015. The maximum GC pause times decrease with the second tuning while we observe shorter execution time, and improved throughput, as expected with this ML model. However, we do not see the same behavior with GB. Nevertheless, both give slightly better maximum pause times compared to the maximum pause times in the default run time windows which is represented by the orange line.

By only optimizing the minimum size of the young generation in iteration 1, the application achieves up to 24.11% shorter execution time with SVR and ET. We observe a similar pattern for P50 and P99 that the benchmark can handle requests faster and become more responsive compared to the default run up to 31.78% and 6.49% better in P50 and P99, respectively for RFR model. Only, P99 of the runs tuned with GPR suggestions seem improved further in the second iteration with 3.46%.

DaCapo/h2:



(a) Results of iteration 1: Optimizing for throughput, only `-XX:G1NewSizePercent` (minimum percentage flag) is set in tuned runs.



(b) Results of iteration 2: Optimizing for latency without impacting improved throughput, both `-XX:G1NewSizePercent` and `-XX:G1MaxNewSizePercent` (minimum and maximum percentage flags) are set in the tuned runs.

Figure 4.5: Results of DaCapo/h2 benchmark: Comparison graphs of the default and tuned runs for program execution time and reported latency metrics 50th and 99th percentiles of the latency by DaCapo/h2 benchmark. The y-axis indicates the performance measurement (execution time, 50th and 99th percentile of the latency from left to right) and the x-axis shows the results from the default run on the left followed by the results of each model that are SVR, GB, RFR, ET and GPR from left to right. Each bar is the median of 20 runs of the same configuration, the black lines represent the median ± the standard deviation of these runs, and the percentages show the change compared to the default result of the related tuned version.

DaCapo/h2 benchmark is a simulation of the H2 database which is an in-memory database designed for embedded use or in client/server mode [40]. Figure 4.5 shows the execution time, P50, and P99 results of the benchmark for two iterations of tuning. With paying attention to the median values of the execution time, there is a maximum improvement of 0.91% in RFR and ET for the first iteration and 1.23% with SVR for the second. If we also consider standard deviations, the reduction in the medians might be negligible.

4.3.2 Comparison of the Overall Performance of ML Models

In this section, we compare five ML models we used in this project: SVR, GB, RFR, ET and GPR. for the benchmarks SPECjbb2015, DaCapo/cassandra and DaCapo/h2, separately. The Tables 4.3, 4.4 and 4.5 contain the following results of the benchmark runs by tuning them with the flag values suggested by the models:

- For throughput: Tables show the changes observed after the first and the second iterations compared to the default run. The positive percentage represents that we improved the throughput by that amount while a negative percentage indicates a decrease in throughput. We monitor max-jOPS for SPECjbb2015 and execution time for DaCapo benchmarks.

- For latency: As the goal of the second iteration is to flatten the peaks of latency-related metrics (maximum pause time for SPECjbb2015; P50 and P99 for DaCapo benchmarks), we record the difference between iteration 1 and iteration 2 to observe to what extent we manage to improve the responsiveness. We calculate this difference as

$$(latency_{iter1} - latency_{iter2})/latency_{iter1} * 100 \tag{4.1}$$

  where $latency_{iter1}$ and $latency_{iter2}$ are the related latency values for each benchmark as the result of iteration 1 and 2. Therefore, if the percentage is positive, this means that we achieved to reduce latency as expected, otherwise, latency is influenced negatively.

In these tables, we consider the median values of the multiple runs of the same configurations for the default and tuned runs since the median value represents the central tendency of the data.

Table 4.3 shows the improvements in throughput of SPECjbb2015 benchmark (the number of max-jOPS) after iterations 1 and 2 for all five models and the difference between the maximum pause GC times of each iteration as a latency indicator. Tables 4.4 and 4.5 illustrate the change in throughput or execution times after iterations 1 and 2 for all five models along with the latency difference between these iterations for DaCapo/cassandra and DaCapo/h2 benchmarks.

Focusing on GPR model results on Table 4.4 as an example, let's explain how to interpret the numbers to understand the performance change. At the

end of the first iteration, throughput improved by 20.6%. Given a tradeoff between throughput and latency, we anticipate observing declined latency after this iteration. Therefore, our objective is to achieve a latency enhancement after the second iteration compared to the initial one while maintaining the improved throughput as much as possible. In this example, we observe that the average latency (P50) of this benchmark worsened by 3.98%, however, we managed to upgrade the tail latency (P99) by 5.14%. The rest of the data can be interpreted based on this explanation by also considering our expectations from the results.

Table 4.3: Improvements for SPECjbb2015: Throughput improvements after iteration 1 and iteration 2 of auto-tuned SPECjbb2015 using suggested configurations by five ML models and the change in responsiveness of the application (maximum GC pause time) between first and second tuning

%		SVR	GB	RFR	ET	GPR
Throughput improvement (max-jOPS)	Iter1	3.49	3.49	2.32	3.49	3.49
	Iter2	3.49	1.16	1.16	1.16	3.49
Latency difference (max GC pause time)	Iter1- Iter2	-31.05	17.11	1.11	-2.77	20.25

4.4 Discussion

The results of SPECjbb2015 benchmark have more significant difference for throughput compared to DaCapo benchmarks. Its workload seems more sensitive to tuning the size of the young generation, which is because larger amounts of short-lived objects. For this benchmark, we can improve the throughput by tuning the minimum young generation usage and control the responsiveness of the program by setting the maximum size of the young generation flag. GB and GPR behave as we aimed by keeping the improved throughput while optimizing the latency in the second iteration.

With the focus on the central tendency of the results – median values of multiple runs, DaCapo/cassandra is improved significantly for both throughput and 50th percentile latency. Although this benchmark simulates a database that is already optimized for latency, we observed that local maximum pause

Table 4.4: Improvements for DaCapo/cassandra: Throughput improvements after iteration 1 and iteration 2 of auto-tuned DaCapo/cassandra using suggested configurations by five ML models and the change in responsiveness of the application (reported P50 – average latency and P99 – tail latency) between first and second tuning

%		SVR	GB	RFR	ET	GPR
Throughput improvement (execution time)	Iter1	24.11	19.12	20.48	24.11	20.6
	Iter2	22.98	21.92	21.42	22.98	20.3
Latency difference (Iter1 - Iter2)	P50	-0.97	-16.82	-6.87	-0.97	-3.98
	P99	-12.51	-6.96	-11.08	-12.51	5.14

Table 4.5: Improvements for DaCapo/h2: Throughput improvements after iteration 1 and iteration 2 of auto-tuned DaCapo/h2 using suggested configurations by five ML models and the change in responsiveness of the application (reported P50 – average latency and P99 – tail latency) between first and second tuning

%		SVR	GB	RFR	ET	GPR
Throughput improvement (execution time)	Iter1	0.05	-0.7	0.91	0.91	0.45
	Iter2	1.23	-0.5	0.76	-0.75	-2.17
Latency difference (Iter1 - Iter2)	P50	18.39	-3.68	-3.73	-49.06	-12.11
	P99	20.9	14.07	-12.89	-49.98	-27.53

times can be slightly optimized, and we obtained better latency values reported by the benchmark with GPR model. Except for GPR on P99, setting the maximum size of the young generation does not have an obvious impact. However, GPR looks promising for this benchmark in terms of reducing the peaks of response times along with better throughput.

For DaCapo/h2, throughput results look similar to the untuned version

of it. The underlying cause may potentially be the fact that this benchmark simulates mostly in-memory database, its performance might be less influenced by JVM memory tuning for young generation options, especially if it is already efficient in managing its memory usage. The results show that SVR achieves slightly optimized throughput along with 20.9% improved latency in 99th percentile after the second iteration.

Overall, the throughput of SPECjbb2015 benchmark reached 3.49% by tuning with the suggested values by GPR in the first iteration and reduced latency while keeping this amount of improved throughput. Also for DaCapo/cassandra, GPR seems more promising than other models. Clearly, it meets our expectations better for latency while achieving 20.3% better throughput. For DaCapo/h2, SVR has the potential to tune the young generation size in Java heap the best among all models for throughput with improved responsiveness as a result of the second iteration.

Chapter 5

Conclusion

In this chapter, we conclude this thesis project by suggesting further research and ending with conclusions.

5.1 Future Work

This thesis proposes the following future work opportunities:

- We have focused on only the GC logs generated by G1 garbage collector. Each garbage collector produce different formatted logs. For the future research, the other garbage collectors such as ZGC, SerialGC and ParallelGC can be explored.
- The label for the ML models in our pipeline is Eden usage before GC events. Our approach can be applied to predict the usage of the Survivor regions, the usage of old generation regions, heap size, and the heap usage. In this way, the pipeline can suggest values for different JVM flags like `-XX:InitialSurvivorRatio` (size of the survivor space), `-XX:NewRatio` (ratio between young and old generations), `-Xmx` and `-Xms` (initial and maximum heap sizes) or more flags can be explored from Java command description guide [8].
- More challenging future research would be to investigate whether one pre-trained ML model might give flag value suggestions to different Java programs for performance optimization purposes instead of training the models for each program individually.

5.2 Conclusion

In this research project, we have established an ML pipeline that learns from the GC log of a single run of a Java program with ML models and suggests the values for young generation size related JVM flags. To have the suggestions, the pipeline is executed separately for each Java program, i.e. the models are trained specifically to each program where we use SPECjbb2015, DaCapo/cassandra, and DaCapo/h2 benchmarks to test our pipeline. With the evaluations and results, we have found that we can improve the throughput with minimal impact on latency depending on the nature of the Java program. In this project, we focused on adjusting two key JVM flags controlling young generation memory within the heap. Such tuning varies widely among Java programs. Although some of them benefit significantly, others demonstrate negligible effects. The reasons for the varying sensitivity of different benchmarks to such tuning remain unclear. However, future research can explore this further using a broader range of Java programs. This may highlight the need for customized optimization approaches that consider the specific characteristics of each Java application and the importance of application-specific JVM configuration in achieving maximum performance improvements. This can be done by improving the strategy of GC log processing and analyzing and extending the type of flags the pipeline can suggest.

References

[1] "Java Software." [Online]. Available: https://www.oracle.com/java/ [Accessed: 2024-01-27] [Page 1.]

[2] "Java Garbage Collection Basics." [Online]. Available: https://www.oracle.com/webfolder/technetwork/tutorials/obe/java/gc01/index.html [Accessed: 2024-01-27] [Pages 1, 7, and 8.]

[3] "Chapter 1. Introduction." [Online]. Available: https://docs.oracle.com/javase/specs/jvms/se7/html/jvms-1.html [Accessed: 2024-01-27] [Page 1.]

[4] "Java® Platform, Standard Edition & Java Development Kit Version 21 API Specification." [Online]. Available: https://docs.oracle.com/en/java/javase/21/docs/specs/man/java.html#enable-logging-with-the-jvm-unified-logging-framework [Accessed: 2024-01-27] [Page 2.]

[5] D. Detlefs, C. Flood, S. Heller, and T. Printezis, "Garbage-first garbage collection," in *Proceedings of the 4th International Symposium on Memory Management*, ser. ISMM '04. New York, NY, USA: Association for Computing Machinery, 2004. doi: 10.1145/1029873.1029879. ISBN 1581139454 p. 37–48. [Online]. Available: https://doi-org.focus.lib.kth.se/10.1145/1029873.1029879 [Page 2.]

[6] "OpenJDK." [Online]. Available: https://openjdk.org/ [Accessed: 2024-05-04] [Page 2.]

[7] "Goal 9 | Department of Economic and Social Affairs." [Online]. Available: https://sdgs.un.org/goals/goal9 [Accessed: 2024-01-29] [Page 4.]

[8] "Java® Platform, Standard Edition & Java Development Kit Version 21 API Specification." [Online]. Available: https://docs.oracle.com/en/jav

a/javase/21/docs/specs/man/java.html#enable-logging-with-the-jvm-unified-logging-framework [Accessed: 2024-04-03] [Pages 5, 10, 20, and 46.]

[9] R. Jones, A. Hosking, and E. Moss, *The Garbage collection handbook : the art of automatic memory management*, second edition. ed. Boca Raton, FL: CRC Press, 2023. ISBN 1-00-327614-8 [Page 7.]

[10] M. Kumar and W. Shi, "Energy Consumption Analysis of Java Command-line Options," in *2019 Tenth International Green and Sustainable Computing Conference (IGSC)*, Oct. 2019. doi: 10.1109/IGSC48788.2019.8957172 pp. 1–8. [Online]. Available: https://ieeexplore.ieee.org/document/8957172 [Accessed: 2024-04-01] [Pages 8 and 18.]

[11] "Garbage-First (G1) Garbage Collector," publisher: September2023. [Online]. Available: https://docs.oracle.com/en/java/javase/21/gctuning/garbage-first-g1-garbage-collector1.html#GUID-F1BE86FA-3EDC-4D4F-BDB4-4B044AD83180 [Accessed: 2024-03-18] [Page 8.]

[12] "Getting Started with the G1 Garbage Collector." [Online]. Available: https://www.oracle.com/technetwork/tutorials/tutorials-1876574.html [Accessed: 2024-05-10] [Page 8.]

[13] "Metaspace - Metaspace - OpenJDK Wiki." [Online]. Available: https://wiki.openjdk.org/display/HotSpot/Metaspace [Accessed: 2024-03-18] [Page 9.]

[14] H. Drucker, C. J. C. Burges, L. Kaufman, A. Smola, and V. Vapnik, "Support Vector Regression Machines," in *Advances in Neural Information Processing Systems*, M. C. Mozer, M. Jordan, and T. Petsche, Eds., vol. 9. MIT Press, 1996. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/1996/file/d38901788c533e8286cb6400b40b386d-Paper.pdf [Page 12.]

[15] J. H. Friedman, "Greedy function approximation: A gradient boosting machine." *The Annals of Statistics*, vol. 29, no. 5, pp. 1189 – 1232, 2001. doi: 10.1214/aos/1013203451 Publisher: Institute of Mathematical Statistics. [Online]. Available: https://doi.org/10.1214/aos/1013203451 [Page 12.]

[16] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 10 2001. doi: 10.1023/A:1010950718922 [Page 12.]

[17] Geurts, P., Ernst, D. & Wehenkel, L., "Extremely randomized trees," pp. 3–42, 2006. doi: 10.1007/s10994-006-6226-1. [Online]. Available: https://doi.org/10.1007/s10994-006-6226-1 [Page 12.]

[18] A. G. Wilson, D. A. Knowles, and Z. Ghahramani, "Gaussian Process Regression Networks," Oct. 2011, arXiv:1110.4411 [q-fin, stat]. [Online]. Available: http://arxiv.org/abs/1110.4411 [Accessed: 2024-02-10] [Page 12.]

[19] "Which Machine Learning Classifiers are Best for Small Datasets?" [Online]. Available: https://www.data-cowboys.com/blog/which-machine-learning-classifiers-are-best-for-small-datasets [Accessed: 2024-03-30] [Page 12.]

[20] M. Ebden, "Gaussian Processes: A Quick Introduction," Aug. 2015, arXiv:1505.02965 [math, stat]. [Online]. Available: http://arxiv.org/abs/1505.02965 [Accessed: 2024-02-18] [Page 12.]

[21] "skopt.BayesSearchCV — scikit-optimize 0.8.1 documentation." [Online]. Available: https://scikit-optimize.github.io/stable/modules/generated/skopt.BayesSearchCV.html [Accessed: 2024-03-18] [Page 13.]

[22] J. Snoek, H. Larochelle, and R. P. Adams, "Practical bayesian optimization of machine learning algorithms," 2012. [Page 13.]

[23] S. Jayasena, M. Fernando, T. Rusira, C. Perera, and C. Philips, "Auto-Tuning the Java Virtual Machine," in *2015 IEEE International Parallel and Distributed Processing Symposium Workshop*, May 2015. doi: 10.1109/IPDPSW.2015.84 pp. 1261–1270. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/7284456 [Accessed: 2024-02-03] [Page 13.]

[24] V. V, P. B. Bindal, D. Singhal, A. V. Subramanyam, and V. Kumar, "OneStopTuner: An End to End Architecture for JVM Tuning of Spark Applications," Sep. 2020, arXiv:2009.06374 [cs]. [Online]. Available: http://arxiv.org/abs/2009.06374 [Accessed: 2024-02-03] [Page 13.]

[25] G. Vijayakumar and R. Bharathi, "Predicting JVM Parameters for Performance Tuning Using Different Regression Algorithms," in *2022 Fourth International Conference on Emerging Research in Electronics, Computer Science and Technology (ICERECT)*, Dec. 2022. doi:

10.1109/ICERECT56837.2022.10060788 pp. 1–8. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10060788 [Accessed: 2024-02-03] [Page 14.]

[26] Oracle Developers, "Automated Tuning of the JVM with Bayesian Optimization," Sep. 2016. [Online]. Available: https://www.youtube.com/watch?v=YhNl468S8CI [Accessed: 2024-02-03] [Page 14.]

[27] P. Lengauer and H. Mössenböck, "The taming of the shrew: increasing performance by automatic parameter tuning for java garbage collectors," in *Proceedings of the 5th ACM/SPEC International Conference on Performance Engineering*, ser. ICPE '14. New York, NY, USA: Association for Computing Machinery, 2014. doi: 10.1145/2568088.2568091. ISBN 9781450327336 p. 111–122. [Online]. Available: https://doi-org.focus.lib.kth.se/10.1145/2568088.2568091 [Pages 14 and 32.]

[28] J. Singer, G. Kovoor, G. Brown, and M. Luján, "Garbage collection auto-tuning for java mapreduce on multi-cores," in *Proceedings of the International Symposium on Memory Management*, ser. ISMM '11. New York, NY, USA: Association for Computing Machinery, 2011. doi: 10.1145/1993478.1993495. ISBN 9781450302630 p. 109–118. [Online]. Available: https://doi-org.focus.lib.kth.se/10.1145/1993478.1993495 [Page 15.]

[29] S. Sahin, W. Cao, Q. Zhang, and L. Liu, "JVM Configuration Management and Its Performance Impact for Big Data Applications | IEEE Conference Publication | IEEE Xplore." doi: 10.1109/BigDataCongress.2016.64. [Online]. Available: https://ieeexplore-ieee-org.focus.lib.kth.se/document/7584970 [Accessed: 2024-04-01] [Page 16.]

[30] S. Cereda, S. Valladares, P. Cremonesi, and S. Doni, "Cgptuner: a contextual gaussian process bandit approach for the automatic tuning of it configurations under varying workload conditions," *Proceedings of the VLDB Endowment*, vol. 14, no. 8, pp. 1401–1413, 2021. [Page 16.]

[31] F. Canales, G. Hecht, and A. Bergel, "Optimization of java virtual machine flags using feature model and genetic algorithm," in *Companion of the ACM/SPEC International Conference on Performance Engineering*, ser. ICPE '21. New York, NY, USA: Association

for Computing Machinery, 2021. doi: 10.1145/3447545.3451177. ISBN 9781450383318 p. 183–186. [Online]. Available: https://doi.org/10.1145/3447545.3451177 [Page 17.]

[32] H. Li, M. Wu, B. Zang, and H. Chen, "Scissorgc: scalable and efficient compaction for java full garbage collection," in *Proceedings of the 15th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, ser. VEE 2019. New York, NY, USA: Association for Computing Machinery, 2019. doi: 10.1145/3313808.3313820. ISBN 9781450360203 p. 108–121. [Online]. Available: https://doi.org/10.1145/3313808.3313820 [Page 20.]

[33] "sklearn.model_selection.train_test_split." [Online]. Available: https://scikit-learn/stable/modules/generated/sklearn.model_selection.train_test_split.html [Accessed: 2024-03-18] [Page 26.]

[34] Ken Stewart, "Mean squared error (MSE) | Definition, Formula, Interpretation, & Facts | Britannica," Apr. 2024. [Online]. Available: https://www.britannica.com/science/mean-squared-error [Accessed: 2024-05-04] [Page 26.]

[35] "SPECjbb®2015." [Online]. Available: https://www.spec.org/jbb2015/ [Accessed: 2024-02-10] [Pages 31 and 32.]

[36] S. M. Blackburn, R. Garner, C. Hoffmann, A. M. Khang, K. S. McKinley, R. Bentzur, A. Diwan, D. Feinberg, D. Frampton, S. Z. Guyer, M. Hirzel, A. Hosking, M. Jump, H. Lee, J. E. B. Moss, A. Phansalkar, D. Stefanović, T. VanDrunen, D. von Dincklage, and B. Wiedermann, "The DaCapo benchmarks: java benchmarking development and analysis," *SIGPLAN notices*, vol. 41, no. 10, pp. 169–190, 2006, publisher: Association for Computing Machinery (ACM). [Pages 31 and 32.]

[37] "spec.org," https://www.spec.org/jbb2015/docs/userguide.pdf, [Accessed 31-03-2024]. [Page 32.]

[38] W. Zhao, S. M. Blackburn, and K. S. McKinley, "Low-latency, high-throughput garbage collection," in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, ser. PLDI 2022. New York, NY, USA: Association for Computing Machinery, 2022. doi: 10.1145/3519939.3523440. ISBN

9781450392655 p. 76–91. [Online]. Available: https://doi.org/10.1145/3519939.3523440 [Page 32.]

[39] “Apache Cassandra | Apache Cassandra Documentation.” [Online]. Available: https://cassandra.apache.org/_/index.html [Accessed: 2024-04-30] [Page 39.]

[40] “H2 Database Engine.” [Online]. Available: https://www.h2database.com/html/main.html [Accessed: 2024-04-30] [Page 41.]

Appendix A

Supporting materials

A.1 Example GC Log Analysis

Example portions of the data we use in this project: a snippet from a raw GC log file, after extracting information from that GC log in tabular format and the data after processing.

A.1.1 Example GC Log

```
[2024-03-22T10:22:02.125-0700][info] Heap region size: 8M
[2024-03-22T10:22:04.165-0700][info] Using G1
[2024-03-22T10:22:04.165-0700][info] Heap address: 0x00000001c0000000, size: 25600 MB, Compressed Oops
      mode: Zero based, Oop shift amount: 3
[2024-03-22T10:22:13.766-0700][info] GC(0) Pause Young (Normal) (G1 Evacuation Pause)
[2024-03-22T10:22:13.767-0700][info] GC(0) Using 33 workers of 33 for evacuation
[2024-03-22T10:22:13.776-0700][info] GC(0)   Pre Evacuate Collection Set: 0.1ms
[2024-03-22T10:22:13.776-0700][info] GC(0)   Evacuate Collection Set: 6.7ms
[2024-03-22T10:22:13.776-0700][info] GC(0)   Post Evacuate Collection Set: 1.5ms
[2024-03-22T10:22:13.776-0700][info] GC(0)   Other: 9.0ms
[2024-03-22T10:22:13.776-0700][info] GC(0) Eden regions: 160->0(159)
[2024-03-22T10:22:13.776-0700][info] GC(0) Survivor regions: 0->1(20)
[2024-03-22T10:22:13.776-0700][info] GC(0) Old regions: 0->0
[2024-03-22T10:22:13.776-0700][info] GC(0) Archive regions: 0->0
[2024-03-22T10:22:13.776-0700][info] GC(0) Humongous regions: 0->0
[2024-03-22T10:22:13.776-0700][info] GC(0) Metaspace: 20576K(20992K)->20576K(20992K) NonClass: 18167K
      (18432K)->18167K(18432K) Class: 2409K(2560K)->2409K(2560K)
[2024-03-22T10:22:13.776-0700][info] GC(0) Pause Young (Normal) (G1 Evacuation Pause) 1280M->5M(25600M
      ) 10.217ms
[2024-03-22T10:22:13.776-0700][info] GC(0) User=0.16s Sys=0.01s Real=0.01s
[2024-03-22T10:22:13.927-0700][info] GC(1) Pause Young (Concurrent Start) (Metadata GC Threshold)
[2024-03-22T10:22:13.927-0700][info] GC(1) Using 33 workers of 33 for evacuation
[2024-03-22T10:22:13.971-0700][info] GC(1)   Pre Evacuate Collection Set: 0.1ms
[2024-03-22T10:22:13.971-0700][info] GC(1)   Evacuate Collection Set: 15.3ms
[2024-03-22T10:22:13.971-0700][info] GC(1)   Post Evacuate Collection Set: 0.7ms
[2024-03-22T10:22:13.971-0700][info] GC(1)   Other: 28.0ms
[2024-03-22T10:22:13.971-0700][info] GC(1) Eden regions: 7->0(159)
[2024-03-22T10:22:13.971-0700][info] GC(1) Survivor regions: 1->1(20)
[2024-03-22T10:22:13.971-0700][info] GC(1) Old regions: 0->0
[2024-03-22T10:22:13.971-0700][info] GC(1) Archive regions: 0->0
[2024-03-22T10:22:13.971-0700][info] GC(1) Humongous regions: 3->3
[2024-03-22T10:22:13.971-0700][info] GC(1) Metaspace: 20823K(21248K)->20823K(21248K) NonClass: 18394K
      (18688K)->18394K(18688K) Class: 2429K(2560K)->2429K(2560K)
[2024-03-22T10:22:13.971-0700][info] GC(1) Pause Young (Concurrent Start) (Metadata GC Threshold) 80M
      ->30M(25600M) 44.115ms
[2024-03-22T10:22:13.971-0700][info] GC(1) User=0.46s Sys=0.00s Real=0.05s
...
```

```
...
[2024-03-22T10:32:00.111-0700][info] GC(99) Pause Young (Normal) (G1 Evacuation Pause)
[2024-03-22T10:32:00.111-0700][info] GC(99) Using 33 workers of 33 for evacuation
[2024-03-22T10:32:00.231-0700][info] GC(99)   Pre Evacuate Collection Set: 0.2ms
[2024-03-22T10:32:00.231-0700][info] GC(99)   Evacuate Collection Set: 114.8ms
[2024-03-22T10:32:00.231-0700][info] GC(99)   Post Evacuate Collection Set: 3.7ms
[2024-03-22T10:32:00.231-0700][info] GC(99)   Other: 1.6ms
[2024-03-22T10:32:00.231-0700][info] GC(99) Eden regions: 1355->0(1361)
[2024-03-22T10:32:00.231-0700][info] GC(99) Survivor regions: 91->91(181)
[2024-03-22T10:32:00.231-0700][info] GC(99) Old regions: 1085->1085
[2024-03-22T10:32:00.231-0700][info] GC(99) Archive regions: 0->0
[2024-03-22T10:32:00.231-0700][info] GC(99) Humongous regions: 167->167
[2024-03-22T10:32:00.231-0700][info] GC(99) Metaspace: 27590K(28544K)->27590K(28544K) NonClass: 24663K
    (25344K)->24663K(25344K) Class: 2927K(3200K)->2927K(3200K)
[2024-03-22T10:32:00.231-0700][info] GC(99) Pause Young (Normal) (G1 Evacuation Pause) 21578M->10738M
    (25600M) 120.261ms
[2024-03-22T10:32:00.231-0700][info] GC(99) User=3.41s Sys=0.01s Real=0.12s
[2024-03-22T10:32:01.971-0700][info] GC(100) Pause Young (Normal) (G1 Evacuation Pause)
[2024-03-22T10:32:01.971-0700][info] GC(100) Using 33 workers of 33 for evacuation
[2024-03-22T10:32:02.082-0700][info] GC(100)   Pre Evacuate Collection Set: 0.2ms
[2024-03-22T10:32:02.082-0700][info] GC(100)   Evacuate Collection Set: 105.2ms
[2024-03-22T10:32:02.082-0700][info] GC(100)   Post Evacuate Collection Set: 3.6ms
[2024-03-22T10:32:02.082-0700][info] GC(100)   Other: 1.7ms
[2024-03-22T10:32:02.082-0700][info] GC(100) Eden regions: 1361->0(1371)
[2024-03-22T10:32:02.082-0700][info] GC(100) Survivor regions: 91->88(182)
[2024-03-22T10:32:02.082-0700][info] GC(100) Old regions: 1085->1085
[2024-03-22T10:32:02.082-0700][info] GC(100) Archive regions: 0->0
[2024-03-22T10:32:02.082-0700][info] GC(100) Humongous regions: 167->167
[2024-03-22T10:32:02.082-0700][info] GC(100) Metaspace: 27590K(28544K)->27590K(28544K) NonClass: 24663
    K(25344K)->24663K(25344K) Class: 2927K(3200K)->2927K(3200K)
[2024-03-22T10:32:02.082-0700][info] GC(100) Pause Young (Normal) (G1 Evacuation Pause) 21626M->10710M
    (25600M) 110.639ms
[2024-03-22T10:32:02.082-0700][info] GC(100) User=3.46s Sys=0.02s Real=0.11s
[2024-03-22T10:32:03.841-0700][info] GC(101) Pause Young (Normal) (G1 Evacuation Pause)
[2024-03-22T10:32:03.841-0700][info] GC(101) Using 33 workers of 33 for evacuation
[2024-03-22T10:32:03.951-0700][info] GC(101)   Pre Evacuate Collection Set: 0.1ms
[2024-03-22T10:32:03.951-0700][info] GC(101)   Evacuate Collection Set: 104.8ms
[2024-03-22T10:32:03.951-0700][info] GC(101)   Post Evacuate Collection Set: 3.5ms
[2024-03-22T10:32:03.951-0700][info] GC(101)   Other: 1.7ms
[2024-03-22T10:32:03.951-0700][info] GC(101) Eden regions: 1371->0(1377)
[2024-03-22T10:32:03.951-0700][info] GC(101) Survivor regions: 88->88(183)
[2024-03-22T10:32:03.951-0700][info] GC(101) Old regions: 1085->1085
[2024-03-22T10:32:03.951-0700][info] GC(101) Archive regions: 0->0
[2024-03-22T10:32:03.951-0700][info] GC(101) Humongous regions: 167->167
[2024-03-22T10:32:03.952-0700][info] GC(101) Metaspace: 27590K(28544K)->27590K(28544K) NonClass: 24663
    K(25344K)->24663K(25344K) Class: 2927K(3200K)->2927K(3200K)
[2024-03-22T10:32:03.952-0700][info] GC(101) Pause Young (Normal) (G1 Evacuation Pause) 21678M->10710M
    (25600M) 110.233ms
[2024-03-22T10:32:03.952-0700][info] GC(101) User=3.20s Sys=0.02s Real=0.11s
[2024-03-22T10:32:05.656-0700][info] GC(102) Pause Young (Normal) (G1 Evacuation Pause)
[2024-03-22T10:32:05.656-0700][info] GC(102) Using 33 workers of 33 for evacuation
[2024-03-22T10:32:05.788-0700][info] GC(102)   Pre Evacuate Collection Set: 0.2ms
[2024-03-22T10:32:05.788-0700][info] GC(102)   Evacuate Collection Set: 126.1ms
[2024-03-22T10:32:05.788-0700][info] GC(102)   Post Evacuate Collection Set: 3.8ms
[2024-03-22T10:32:05.788-0700][info] GC(102)   Other: 1.4ms
[2024-03-22T10:32:05.788-0700][info] GC(102) Eden regions: 1377->0(1360)
[2024-03-22T10:32:05.788-0700][info] GC(102) Survivor regions: 88->100(184)
[2024-03-22T10:32:05.788-0700][info] GC(102) Old regions: 1085->1085
[2024-03-22T10:32:05.788-0700][info] GC(102) Archive regions: 0->0
[2024-03-22T10:32:05.788-0700][info] GC(102) Humongous regions: 167->167
[2024-03-22T10:32:05.788-0700][info] GC(102) Metaspace: 27590K(28544K)->27590K(28544K) NonClass: 24663
    K(25344K)->24663K(25344K) Class: 2927K(3200K)->2927K(3200K)
[2024-03-22T10:32:05.788-0700][info] GC(102) Pause Young (Normal) (G1 Evacuation Pause) 21726M->10812M
    (25600M) 131.558ms
[2024-03-22T10:32:05.788-0700][info] GC(102) User=4.12s Sys=0.06s Real=0.14s
[2024-03-22T10:32:07.484-0700][info] GC(103) Pause Young (Normal) (G1 Evacuation Pause)
[2024-03-22T10:32:07.484-0700][info] GC(103) Using 33 workers of 33 for evacuation
[2024-03-22T10:32:07.606-0700][info] GC(103)   Pre Evacuate Collection Set: 0.2ms
[2024-03-22T10:32:07.606-0700][info] GC(103)   Evacuate Collection Set: 116.5ms
[2024-03-22T10:32:07.606-0700][info] GC(103)   Post Evacuate Collection Set: 3.5ms
[2024-03-22T10:32:07.606-0700][info] GC(103)   Other: 1.7ms
[2024-03-22T10:32:07.606-0700][info] GC(103) Eden regions: 1360->0(1371)
[2024-03-22T10:32:07.606-0700][info] GC(103) Survivor regions: 100->91(183)
[2024-03-22T10:32:07.606-0700][info] GC(103) Old regions: 1085->1087
[2024-03-22T10:32:07.606-0700][info] GC(103) Archive regions: 0->0
[2024-03-22T10:32:07.606-0700][info] GC(103) Humongous regions: 167->167
[2024-03-22T10:32:07.606-0700][info] GC(103) Metaspace: 27590K(28544K)->27590K(28544K) NonClass: 24663
    K(25344K)->24663K(25344K) Class: 2927K(3200K)->2927K(3200K)
[2024-03-22T10:32:07.606-0700][info] GC(103) Pause Young (Normal) (G1 Evacuation Pause) 21692M->10750M
    (25600M) 121.987ms
```

```
[2024-03-22T10:32:07.606-0700][info] GC(103) User=3.82s Sys=0.04s Real=0.12s
[2024-03-22T10:32:09.354-0700][info] GC(104) Pause Young (Normal) (G1 Evacuation Pause)
[2024-03-22T10:32:09.354-0700][info] GC(104) Using 33 workers of 33 for evacuation
[2024-03-22T10:32:09.473-0700][info] GC(104)   Pre Evacuate Collection Set: 0.2ms
[2024-03-22T10:32:09.473-0700][info] GC(104)   Evacuate Collection Set: 112.4ms
[2024-03-22T10:32:09.473-0700][info] GC(104)   Post Evacuate Collection Set: 3.8ms
[2024-03-22T10:32:09.473-0700][info] GC(104)   Other: 2.6ms
[2024-03-22T10:32:09.473-0700][info] GC(104) Eden regions: 1371->0(1373)
[2024-03-22T10:32:09.473-0700][info] GC(104) Survivor regions: 91->92(183)
[2024-03-22T10:32:09.473-0700][info] GC(104) Old regions: 1087->1087
[2024-03-22T10:32:09.473-0700][info] GC(104) Archive regions: 0->0
[2024-03-22T10:32:09.473-0700][info] GC(104) Humongous regions: 167->167
[2024-03-22T10:32:09.473-0700][info] GC(104) Metaspace: 27590K(28544K)->27590K(28544K) NonClass: 24663
    K(25344K)->24663K(25344K) Class: 2927K(3200K)->2927K(3200K)
[2024-03-22T10:32:09.473-0700][info] GC(104) Pause Young (Normal) (G1 Evacuation Pause) 21718M->10758M
    (25600M) 119.037ms
[2024-03-22T10:32:09.473-0700][info] GC(104) User=3.67s Sys=0.06s Real=0.12s
[2024-03-22T10:32:11.213-0700][info] GC(105) Pause Young (Normal) (G1 Evacuation Pause)
[2024-03-22T10:32:11.213-0700][info] GC(105) Using 33 workers of 33 for evacuation
[2024-03-22T10:32:11.331-0700][info] GC(105)   Pre Evacuate Collection Set: 2.6ms
[2024-03-22T10:32:11.332-0700][info] GC(105)   Evacuate Collection Set: 110.7ms
[2024-03-22T10:32:11.332-0700][info] GC(105)   Post Evacuate Collection Set: 3.4ms
[2024-03-22T10:32:11.332-0700][info] GC(105)   Other: 2.0ms
[2024-03-22T10:32:11.332-0700][info] GC(105) Eden regions: 1373->0(1370)
[2024-03-22T10:32:11.332-0700][info] GC(105) Survivor regions: 92->95(184)
[2024-03-22T10:32:11.332-0700][info] GC(105) Old regions: 1087->1087
[2024-03-22T10:32:11.332-0700][info] GC(105) Archive regions: 0->0
[2024-03-22T10:32:11.332-0700][info] GC(105) Humongous regions: 167->167
[2024-03-22T10:32:11.332-0700][info] GC(105) Metaspace: 27590K(28544K)->27590K(28544K) NonClass: 24663
    K(25344K)->24663K(25344K) Class: 2927K(3200K)->2927K(3200K)
[2024-03-22T10:32:11.332-0700][info] GC(105) Pause Young (Normal) (G1 Evacuation Pause) 21742M->10782M
    (25600M) 118.768ms
[2024-03-22T10:32:11.332-0700][info] GC(105) User=3.63s Sys=0.04s Real=0.12s
[2024-03-22T10:32:13.044-0700][info] GC(106) Pause Young (Normal) (G1 Evacuation Pause)
[2024-03-22T10:32:13.044-0700][info] GC(106) Using 33 workers of 33 for evacuation
[2024-03-22T10:32:13.150-0700][info] GC(106)   Pre Evacuate Collection Set: 0.2ms
[2024-03-22T10:32:13.150-0700][info] GC(106)   Evacuate Collection Set: 100.1ms
[2024-03-22T10:32:13.150-0700][info] GC(106)   Post Evacuate Collection Set: 3.7ms
[2024-03-22T10:32:13.150-0700][info] GC(106)   Other: 1.7ms
[2024-03-22T10:32:13.150-0700][info] GC(106) Eden regions: 1370->0(1379)
[2024-03-22T10:32:13.150-0700][info] GC(106) Survivor regions: 95->90(184)
[2024-03-22T10:32:13.150-0700][info] GC(106) Old regions: 1087->1087
[2024-03-22T10:32:13.150-0700][info] GC(106) Archive regions: 0->0
[2024-03-22T10:32:13.150-0700][info] GC(106) Humongous regions: 167->167
[2024-03-22T10:32:13.150-0700][info] GC(106) Metaspace: 27590K(28544K)->27590K(28544K) NonClass: 24663
    K(25344K)->24663K(25344K) Class: 2927K(3200K)->2927K(3200K)
[2024-03-22T10:32:13.150-0700][info] GC(106) Pause Young (Normal) (G1 Evacuation Pause) 21742M->10742M
    (25600M) 105.702ms
[2024-03-22T10:32:13.150-0700][info] GC(106) User=3.32s Sys=0.00s Real=0.11s
[2024-03-22T10:32:14.950-0700][info] GC(107) Pause Young (Normal) (G1 Evacuation Pause)
[2024-03-22T10:32:14.950-0700][info] GC(107) Using 33 workers of 33 for evacuation
[2024-03-22T10:32:15.055-0700][info] GC(107)   Pre Evacuate Collection Set: 0.2ms
[2024-03-22T10:32:15.055-0700][info] GC(107)   Evacuate Collection Set: 99.6ms
[2024-03-22T10:32:15.055-0700][info] GC(107)   Post Evacuate Collection Set: 3.5ms
[2024-03-22T10:32:15.055-0700][info] GC(107)   Other: 2.1ms
[2024-03-22T10:32:15.055-0700][info] GC(107) Eden regions: 1379->0(1387)
[2024-03-22T10:32:15.055-0700][info] GC(107) Survivor regions: 90->87(184)
[2024-03-22T10:32:15.055-0700][info] GC(107) Old regions: 1087->1087
[2024-03-22T10:32:15.055-0700][info] GC(107) Archive regions: 0->0
[2024-03-22T10:32:15.055-0700][info] GC(107) Humongous regions: 167->167
[2024-03-22T10:32:15.055-0700][info] GC(107) Metaspace: 27590K(28544K)->27590K(28544K) NonClass: 24663
    K(25344K)->24663K(25344K) Class: 2927K(3200K)->2927K(3200K)
[2024-03-22T10:32:15.056-0700][info] GC(107) Pause Young (Normal) (G1 Evacuation Pause) 21774M->10722M
    (25600M) 105.478ms
[2024-03-22T10:32:15.056-0700][info] GC(107) User=3.29s Sys=0.02s Real=0.10s
...
...
...
[2024-03-22T12:45:11.602-0700][info] GC(2019) User=2.45s Sys=0.24s Real=0.08s
[2024-03-22T12:45:23.165-0700][info] Heap
[2024-03-22T12:45:23.165-0700][info]  garbage-first heap   total 26214400K, used 13677337K [0
    x00000001c0000000, 0x0000000800000000)
[2024-03-22T12:45:23.165-0700][info]   region size 8192K, 706 young (5783552K), 67 survivors (548864K)
[2024-03-22T12:45:23.165-0700][info]  Metaspace       used 37732K, capacity 38986K, committed 39296K,
    reserved 1083392K
[2024-03-22T12:45:23.165-0700][info]   class space    used 3941K, capacity 4415K, committed 4480K,
    reserved 1048576K
```

A.1.2 Example Regular Expressions Used to Extract Features From GC Logs

```
Timestamp = "(?P<Timestamp>\d+\-\d+\-\d+\w+:\d+:\d+\.\d+[\+\-]\d+):\s+"
PauseType = "\s*\((?P<PauseType>[\w\s]+)\)"
HeapUsageBefore = "\s+(?P<HeapUsageBefore>\d+)"
HeapUsageBeforeUnit = "(?P<HeapUsageBeforeUnit>[a-zA-Z])->"
HeapUsageAfter = "(?P<HeapUsageAfter>\d+)"
HeapUsageAfterUnit = "(?P<HeapUsageAfterUnit>[a-zA-Z])"
HeapSize = "\((?P<HeapSize>\d+)"
HeapSizeUnit = "(?P<HeapSizeUnit>[a-zA-Z])\)"
PauseTime = ",\s+(?P<PauseTime>\d+[.,]\d+)"
PauseTimeUnit = "\s+(?P<PauseTimeUnit>\w+)\]"
```

A.1.3 Extracted Features From GC Log in Tabular Format

Table A.1 shows an example portion of the extracted features from a GC log file. Due to its length, we present a truncated version for visualization purposes. "..." indicates the additional existing rows or columns. All the features, i.e., the column names, are listed below with their units:

- Timestamps, in date-time format and converted to seconds starting from 0.0 seconds (second)
- GCID, the ID of the GC events starting from 0
- Usage of heap before and after GC events and allocated heap size (megabyte)
- Usage of young generation (Eden and survivor regions separately) before and after GC events and allocated young generation size (megabyte)
- Usage of old generation before and after GC events and allocated old generation size (megabyte)
- Usage of metaspace and allocated size before and after GC events (kilobyte)
- Pause time, time spent for each GC event (millisecond)
- Pause types (categorical); concurrent start, mixed, premixed, and normal pauses

Table A.1: An example portion of extracted features from GC log

	Timestamp	GCID	HeapUsage BeforeGC	HeapUsage AfterGC	PauseTime	MetaspaceUsage BeforeGC	EdenUsage BeforeGC	EdenSize	SurvivorsUsage BeforeGC	SurvivorsUsage AfterGC	Survivors Size	OldGenUsage BeforeGC	OldGenUsage AfterGC
0	0.0	0	1280.0	5.0	10.2	20576.0	1280.0	1272.0	0.0	8.0	160.0	0.0	0.0
1	0.2	1	80.0	30.0	44.1	20823.0	56.0	1272.0	8.0	8.0	160.0	0.0	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
100	586.5	99	21578.0	10738.0	120.3	27590.0	10840.0	10888.0	728.0	728.0	1448.0	8680.0	8680.0
101	588.3	100	21626.0	10710.0	110.6	27590.0	10888.0	10968.0	728.0	704.0	1456.0	8680.0	8680.0
102	590.2	101	21678.0	10710.0	110.2	27590.0	10968.0	11016.0	704.0	704.0	1464.0	8680.0	8680.0
103	592.0	102	21726.0	10812.0	131.6	27590.0	11016.0	10880.0	704.0	800.0	1472.0	8680.0	8680.0
104	593.8	103	21692.0	10750.0	122.0	27590.0	10880.0	10968.0	800.0	728.0	1464.0	8680.0	8696.0
105	595.7	104	21718.0	10758.0	119.0	27590.0	10968.0	10984.0	728.0	736.0	1464.0	8696.0	8696.0
106	597.6	105	21742.0	10782.0	118.8	27590.0	10984.0	10960.0	736.0	760.0	1472.0	8696.0	8696.0
107	599.4	106	21742.0	10742.0	105.7	27590.0	10960.0	11032.0	760.0	720.0	1472.0	8696.0	8696.0
108	601.3	107	21774.0	10722.0	105.5	27590.0	11032.0	11096.0	720.0	696.0	1472.0	8696.0	8696.0
109	603.1	108	21818.0	10774.0	119.3	27590.0	11096.0	11032.0	696.0	752.0	1480.0	8696.0	8696.0
110	605.0	109	21806.0	10785.0	134.6	27590.0	11032.0	11008.0	752.0	760.0	1480.0	8696.0	8696.0
111	606.8	110	21793.0	10787.0	148.1	27590.0	11008.0	11000.0	760.0	760.0	1472.0	8696.0	8696.0
112	608.6	111	21787.0	10769.0	120.7	27590.0	11000.0	11016.0	760.0	736.0	1472.0	8696.0	8704.0
113	610.4	112	21785.0	10787.0	119.4	27590.0	11016.0	11000.0	736.0	752.0	1472.0	8704.0	8704.0
114	612.3	113	21787.0	10773.0	108.3	27590.0	11000.0	11032.0	752.0	736.0	1472.0	8704.0	8704.0
115	614.3	114	21805.0	10775.0	112.3	27590.0	11032.0	11032.0	736.0	736.0	1472.0	8704.0	8704.0
116	616.1	115	21807.0	10745.0	103.4	27590.0	11032.0	11088.0	736.0	704.0	1472.0	8704.0	8712.0
117	618.0	116	21833.0	10827.0	141.7	27590.0	11088.0	10968.0	704.0	784.0	1480.0	8712.0	8712.0
118	619.9	117	21795.0	10818.0	120.6	27590.0	10968.0	10960.0	784.0	776.0	1472.0	8712.0	8712.0
119	621.7	118	21778.0	10788.0	116.9	27591.0	10960.0	11000.0	776.0	744.0	1472.0	8712.0	8720.0
120	623.5	119	21788.0	10818.0	114.4	27591.0	11000.0	10952.0	744.0	768.0	1472.0	8720.0	8720.0
121	625.3	120	21770.0	10788.0	109.1	27591.0	10952.0	11000.0	768.0	736.0	1472.0	8720.0	8720.0
122	627.1	121	21788.0	10761.0	108.0	27591.0	11000.0	11048.0	736.0	712.0	1472.0	8720.0	8720.0
123	629.0	122	21809.0	10871.0	128.7	27591.0	11048.0	10880.0	712.0	816.0	1472.0	8720.0	8720.0
124	630.7	123	21751.0	10820.0	124.1	27591.0	10880.0	10920.0	816.0	744.0	1464.0	8720.0	8744.0
125	632.6	124	21740.0	10768.0	109.7	27591.0	10920.0	11000.0	744.0	696.0	1464.0	8744.0	8744.0
126	634.5	125	21768.0	10828.0	114.2	27591.0	11000.0	10928.0	696.0	752.0	1464.0	8744.0	8744.0
127	636.2	126	21756.0	10808.0	107.9	27591.0	10928.0	10952.0	752.0	736.0	1464.0	8744.0	8744.0
128	638.0	127	21760.0	10756.0	116.2	27591.0	10952.0	11048.0	736.0	680.0	1464.0	8744.0	8744.0
129	639.9	128	21804.0	10812.0	119.4	27591.0	11048.0	10984.0	680.0	736.0	1472.0	8744.0	8744.0
130	641.7	129	21796.0	10816.0	115.9	27591.0	10984.0	10976.0	736.0	744.0	1472.0	8744.0	8744.0
131	643.6	130	21792.0	10820.0	119.7	27591.0	10976.0	10968.0	744.0	744.0	1472.0	8744.0	8744.0
132	645.4	131	21788.0	10829.0	126.8	27591.0	10968.0	10944.0	744.0	752.0	1464.0	8744.0	8744.0
133	647.1	132	21773.0	10811.0	115.3	27591.0	10944.0	10976.0	752.0	736.0	1464.0	8744.0	8744.0
134	648.9	133	21787.0	10836.0	125.3	27592.0	10976.0	10936.0	736.0	760.0	1464.0	8744.0	8752.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...	...	...	...	...	...
2022	8571.7	2018	15810.0	8207.0	87.8	27786.0	7664.0	8536.0	440.0	504.0	1016.0	6400.0	6400.0
2023	8577.8	2019	16743.0	8244.0	83.8	27789.0	8536.0	10288.0	504.0	536.0	1136.0	6400.0	6400.0

A.1.4 Data in Tabular Format After Processing

Table A.2 shows an example portion of the processed data. Due to its length, we present a truncated version for visualization purposes. ”...” indicates the additional existing rows or columns. All the features, i.e., the column names, are listed below with their units:

- Time window start second (second)
- Net time (%)
- GC frequency, the number of GC events within a time window
- Total and maximum GC pause time in each time window (millisecond)
- 50th and 90th percentile of the pause times (millisecond)
- Usage of heap, metaspace, Eden, survivor, old region; for all before and after GC events and their allocated sizes (megabyte)
- Allocation rate3.1 (megabyte)
- Pause types (boolean); concurrent start, mixed, premixed, and normal pauses

Table A.2: An example portion of processed dataset

	Time Window Start Second	Net Time	GC Fre-quency	Maximum GC Pause Time	HeapUsage BeforeGC	HeapUsage AfterGC	EdenUsage BeforeGC	SurvivorsUsage BeforeGC	...	OldGenUsage BeforeGC	OldGenUsage AfterGC	Allocation Rate	PauseType Concurrent Start	PauseType Normal
0	0.000	0.992	8	788.033	2191.120	1488.120	1031.000	97.000	...	902.000	1261.000	311.660	True	True
1	38.090	0.996	6	202.906	3680.500	2807.170	1126.670	153.330	...	2081.330	2416.000	252.040	False	True
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
100	3809.000	0.983	33	122.768	18919.330	4199.210	14720.480	639.520	...	3270.790	3271.270	2520.650	False	True
101	3847.090	0.983	33	113.349	18923.580	4203.090	14720.730	639.270	...	3274.180	3274.910	2559.760	False	True
102	3885.180	0.983	34	106.821	18927.620	4207.940	14721.180	638.820	...	3278.590	3279.060	2602.850	False	True
103	3923.270	0.983	34	106.821	18931.760	4212.560	14720.470	639.530	...	3281.880	3282.350	2641.640	False	True
104	3961.360	0.982	35	118.969	18935.600	4218.660	14717.710	642.290	...	3286.170	3286.630	2685.360	False	True
105	3999.450	0.982	35	118.969	18939.510	4223.940	14717.030	642.970	...	3289.370	3289.830	2734.200	False	True
106	4037.540	0.981	36	118.969	18943.810	4228.440	14716.000	644.000	...	3294.220	3294.670	2783.570	False	True
107	4075.630	0.981	36	118.969	18947.420	4230.530	14716.890	643.110	...	3297.330	3297.780	2832.160	False	True
108	4113.720	0.981	37	118.969	18951.780	4236.570	14716.540	643.460	...	3302.050	3302.700	2879.190	False	True
109	4151.810	0.980	38	110.408	18956.050	4242.240	14714.320	645.680	...	3305.890	3306.530	2915.290	False	True
110	4189.900	0.980	38	111.347	18959.970	4246.000	14713.680	646.320	...	3310.320	3310.950	2927.890	False	True
111	4227.990	0.980	38	111.347	18964.340	4252.630	14712.840	647.160	...	3314.530	3315.160	2948.090	False	True
112	4266.080	0.980	39	111.347	18968.900	4258.510	14711.380	648.620	...	3319.180	3319.790	2964.260	False	True
113	4304.170	0.980	39	111.347	18973.560	4263.870	14710.770	649.230	...	3323.690	3324.310	2969.380	False	True
114	4342.260	0.980	39	111.347	18978.280	4267.360	14711.380	648.620	...	3328.210	3328.620	2993.640	False	True
115	4380.350	0.980	39	102.955	18982.510	4271.510	14712.210	647.790	...	3332.310	3332.920	3020.070	False	True
116	4418.440	0.980	40	101.227	18987.530	4277.650	14710.400	649.600	...	3337.000	3337.600	3059.900	False	True
117	4456.530	0.980	40	104.545	18992.080	4282.050	14710.200	649.800	...	3341.800	3342.400	3087.810	False	True
118	4494.620	0.980	40	108.058	18996.350	4287.820	14709.000	651.000	...	3346.000	3346.600	3134.020	False	True
119	4532.710	0.979	40	108.058	19000.720	4294.270	14708.200	651.800	...	3350.800	3351.400	3164.570	False	True
120	4570.800	0.978	42	108.058	19005.050	4301.120	14704.760	655.240	...	3355.050	3355.620	3204.880	False	True
121	4608.890	0.979	41	108.058	19009.490	4305.560	14705.170	654.830	...	3359.800	3360.200	3232.230	False	True
122	4646.980	0.978	42	108.058	19014.120	4312.000	14702.670	657.330	...	3364.380	3364.950	3269.920	False	True
123	4685.070	0.978	43	106.812	19019.050	4316.810	14701.770	658.230	...	3369.120	3369.670	3291.240	False	True
124	4723.160	0.977	44	106.812	19023.730	4322.820	14700.910	659.090	...	3373.820	3374.360	3332.410	False	True
125	4761.250	0.977	44	106.812	19028.570	4326.680	14702.180	657.820	...	3378.730	3379.270	3362.780	False	True
126	4799.340	0.976	45	109.985	19033.490	4333.240	14700.620	659.380	...	3383.290	3383.820	3399.820	False	True
127	4837.430	0.976	45	109.985	19038.400	4338.310	14701.330	658.670	...	3388.270	3388.800	3429.950	False	True
128	4875.520	0.976	45	124.075	19043.000	4344.330	14699.910	660.090	...	3393.240	3393.780	3473.130	False	True
129	4913.610	0.976	45	124.075	19047.820	4350.160	14698.130	661.870	...	3398.040	3398.580	3505.550	False	True
130	4951.700	0.975	46	124.075	19053.070	4361.850	14692.700	667.300	...	3403.480	3404.000	3547.510	False	True
131	4989.790	0.975	46	124.075	19057.850	4369.520	14688.870	671.130	...	3408.350	3409.040	3589.990	False	True
132	5027.880	0.974	47	124.075	19063.000	4375.640	14688.510	671.490	...	3413.450	3413.960	3623.010	False	True
133	5065.970	0.974	47	116.355	19068.190	4383.890	14685.280	674.720	...	3418.550	3419.060	3656.490	False	True
134	5104.060	0.974	48	116.355	19073.380	4389.600	14684.500	675.500	...	3423.670	3424.330	3689.800	False	True
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
224	8532.160	0.993	17	236.482	10322.120	7604.820	2871.060	217.410	...	5947.760	6080.000	1269.350	False	True
225	8570.250	0.999	2	87.813	16276.500	8225.500	8100.000	472.000	...	6400.000	6400.000	1385.410	False	True

A.2 Young Generation Size Graphs

The FIgures A.1, A.2 and A.3 in this section show the size of the young generation (Eden and survivors' sizes) before GC events. The first plot is Eden, and the second is survivors size. The scale of y-axes is different, the Eden size is considerably larger than survivors size.

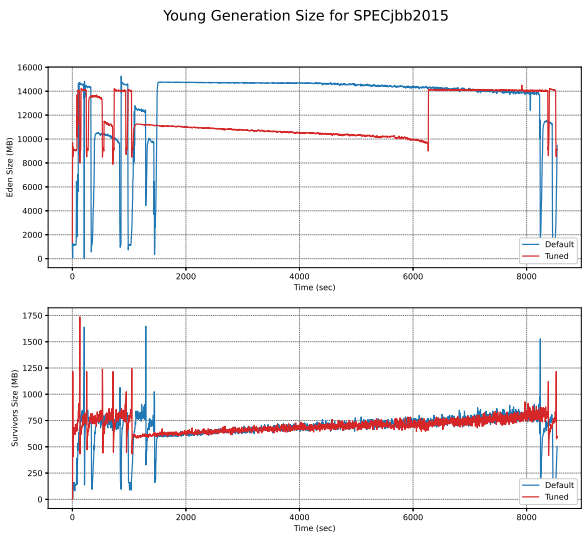


Figure A.1: Graph of SPECjbb2015 young generation size over time: Young generation consists of Eden (first plot) and Survivors regions (second plot). The blue line is young generation size in default run, red line is Survivors size.

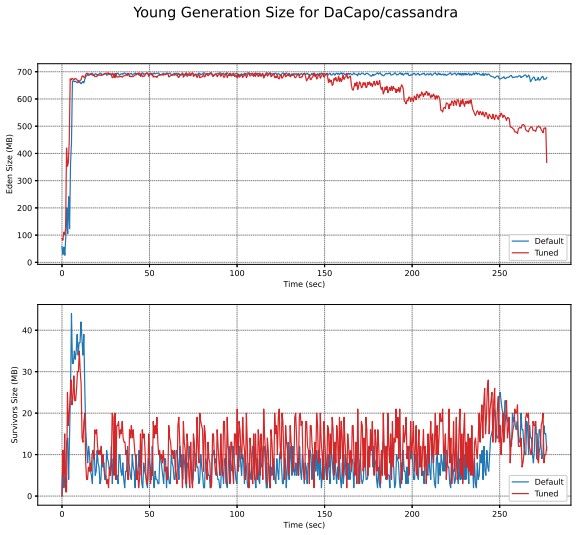


Figure A.2: Graph of DaCapo/cassandra young generation size over time: Young generation consists of Eden (first plot) and Survivors regions (second plot). The blue line is young generation size in default run, red line is Survivors size.

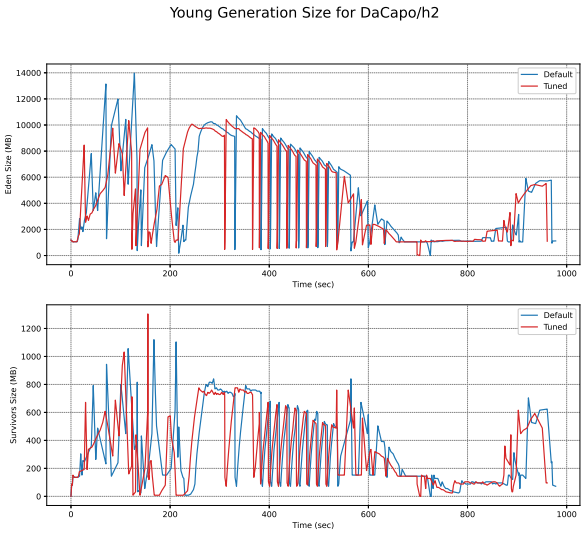


Figure A.3: Graph of DaCapo/h2 young generation size over time: Young generation consists of Eden (first plot) and Survivors regions (second plot). The blue line is young generation size in default run, red line is Survivors size.

A.3 Suggested Flag Values

Table A.3: Suggested flag values by ML models for each benchmark: The flag names are shortened as Min = `-XX:G1NewSizePercent`, Max = `-XX:G1MaxNewSizePercent`

	SPECjbb2015		**DaCapo/cassandra**		**DaCapo/h2**	
Model	Min	Max	Min	Max	Min	Max
SVR	38	60	7	60	4	47
GB	31	60	16	60	6	47
RFR	27	56	32	60	5	44
ET	34	57	7	60	5	47
GPR	38	58	10	60	3	46

A.4 Benchmark Execution Commands

In this section, we provide the commands used to execute benchmarks. The JVM flags we used and benchmark configurations are provided.

Listing A.1: Command to run SPECjbb2015 benchmark

```
<path-to-java> -Xlog:gc*:"gc_file.log":time,level -Xmx${
    heapsize} -Xms${heapsize} -XX:+
    UnlockExperimentalVMOptions -XX:G1MaxNewSizePercent=
    $maxpercent -XX:G1NewSizePercent=$minpercent -XX:+
    DisableExplicitGC -XX:+AlwaysPreTouch -jar
    specjbb2015.jar -m COMPOSITE
```

Listing A.2: Command to run DaCapo/cassandra benchmark

```
<path-to-java> -Xlog:gc*:"gc_file.log":time,level -Xmx${
    heapsize}m -Xms${heapsize}m -XX:+
    UnlockExperimentalVMOptions -XX:G1MaxNewSizePercent=
```

```
$maxpercent -XX:G1NewSizePercent=$minpercent -XX:+
DisableExplicitGC -XX:+AlwaysPreTouch -jar dacapo
-23.11-chopin.jar cassandra -s large > report.txt
```

Listing A.3: Command to run DaCapo/h2 benchmark

```
<path-to-java> -Xlog:gc*:"gc_file.log":time,level -Xmx${
    heapsize}m -Xms${heapsize}m -XX:+
    UnlockExperimentalVMOptions -XX:G1MaxNewSizePercent=
    $maxpercent -XX:G1NewSizePercent=$minpercent -XX:+
    DisableExplicitGC -XX:+AlwaysPreTouch -jar dacapo
    -23.11-chopin.jar h2 -s vlarge > report.txt
```

A.5 The Five Largest Pause Times of Tuned SPECjbb2015 Executions

Table A.4: The five highest maximum GC pause times: The maximum GC pause times of the tuned runs of SPECjbb2015 benchmark that are executed with the suggested flag values by SVR, RFR and ET models. The medians of the top 5 maximum pause times of the overall programs of the same configurations are shown. (iteration 1 depicts the scenario where only the minimum percentage of young generation usage flag is set, while iteration 2 illustrates the situation where both maximum and minimum percentage flags are set concurrently).

	SVR		RFR		ET	
	iter 1	iter 2	iter 1	iter 2	iter 1	iter 2
Top five largest pause times (msec)	1695.69	2222.21	1456.28	1440.08	1451.67	1491.84
	1324.57	1461.51	1263.01	1057.79	1041.36	1139.60
	1133.38	1250.33	1105.96	1032.98	864.10	773.59
	829.85	1055.59	873.67	648.16	857.32	716.09
	688.96	743.16	774.50	604.30	738.37	591.37

TRITA – EECS-EX-2024:631
Stockholm, Sverige 2024

www.kth.se